Fault-Tolerant Control of a Quadrotor After Complete Rotor Loss

Residual Koopman MPC and Fault-Detection-Delay Limits

EE650 Basics of Modern Control Systems: Detailed Project Report

Pranshu Kumar (220799) Mudita Jain (220675)
Prashant Gupta (220803) Aarush Singh Kushwaha (230019)

October 2026

Abstract

When one rotor of a quadrotor (a drone with four propellers) stops completely, the drone can no longer hover level. The only steady way it can still fly is to spin continuously about a slightly tilted axis. Mueller and D'Andrea called this state the *relaxed hover* and showed that it can be controlled. This project studies, in simulation, how to bring a drone into this spinning state reliably after a rotor fails.

We ask two connected questions. **First**, the physics model of the spinning drone is good but not perfect: can we keep it and learn only its error from data, with the tools of Koopman methods, and does this predict and control better than the physics model alone or a Koopman model of the full dynamics? **Second**, a fault detector needs time to notice the failure: how long may this take before a safe recovery is no longer possible, and can such a limit be guaranteed mathematically?

Our simulator of the Mueller–D'Andrea drone reproduces their two-rotor and three-rotor-loss equilibria to every published digit, five hover solutions of their more detailed model within 4%, and the spin-up time of their flight within about 4%. For prediction horizons from 100 ms to 1 s, the model that learns only the error (the *residual* model) is the most accurate predictor on the plant it was trained on, and with 10 training episodes it beats a full Koopman model trained on 40. Inside model predictive control (MPC) it reduces a steady altitude error caused by model mismatch by about one third. The learned models do not yet work on a plant with different parameters. In our simulation, the published recovery strategy recovers safely with probability above 0.9 only if the failure is detected within about 12 ms, and below 0.5 beyond about 19 ms, while published detectors need about 20 to 130 ms. A first certificate, valid for the linearized model with thrust limits and based on linear matrix inequalities, covers 22% of the way from the spinning equilibrium to the state right after a failure.

Technical terms are explained where they first appear and are collected in a glossary (Appendix A). Each method is described in three layers: the idea in plain words, the mathematics, and how it is implemented in our code.

Contents

1 Introduction **4**
1.1 How a quadrotor flies 4
1.2 What happens when a rotor fails 4
1.3 The two problems this project studies 4
1.4 Research questions 5
1.5 What we did 6
1.6 Main results in one table 7

2 Background: the ideas we use **7**
2.1 Systems, states, inputs and outputs . . . 7
2.2 Equilibria and linearization . . . 7
2.3 Eigenvalues and stability . . . 8
2.4 Numerical simulation . . . 8
2.5 Feedback control and LQR . . . 8
2.6 Controllability and observability . . . 9
2.7 Model predictive control (MPC) . . . 9
2.8 Saturation and delay . . . 9
2.9 Koopman operators and EDMD . . . 9
2.10 Regression, ridge regression and residual learning . . . 10
2.11 Lyapunov functions and linear matrix inequalities . . . 10
2.12 Statistics used . . . 10

3 Related work: what the papers did **10**
3.1 The three papers we validate against . . . 10
3.2 Other controllers for complete rotor loss . . . 12
3.3 Guarantees and detection delay . . . 12
3.4 Koopman models and residual learning . . . 13
3.5 Tools . . . 14
3.6 The gap . . . 14

4 The mathematical model **14**
4.1 Frames, layout and sign conventions . . . 14
4.2 The state . . . 15
4.3 Thrust, propeller speed and the motors . . . 15
4.4 Translation: Newton's second law . . . 16
4.5 Rotation: Euler's equation with propellers . . . 16
4.6 Attitude . . . 17
4.7 The reduced attitude . . . 17
4.8 The relaxed hover (spinning equilibrium) . . . 17
4.9 Linear models . . . 18
4.10 Truth plants for learning . . . 19

5 How the model is coded **19**
5.1 Package layout . . . 20
5.2 Parameters . . . 20
5.3 Equations of motion . . . 21
5.4 Integration and the sampled controller . . . 21
5.5 Equilibrium solver . . . 22
5.6 Linear models . . . 22
5.7 Reproducibility . . . 22

6 How we validated the simulator **22**
6.1 Layer 1: physics tests against hand-derived results . . . 23
6.2 Layer 2: published numbers . . . 23
6.3 Layer 3: flight data . . . 26
6.4 Summary of the validation . . . 26

7 Linear analysis and LQR controllers **27**
7.1 Controllability: can three rotors steer the failed drone? . . . 27
7.2 Observability: what can the sensors see? . . . 27
7.3 LQR controllers . . . 28

8 Learning the model error: residual Koopman models (RQ1) 29
8.1 Hypotheses . . . 29
8.2 Data . . . 30
8.3 The three predictors . . . 30
8.4 How prediction error is measured . . . 31
8.5 Results . . . 32
8.6 What this means for RQ1 . . . 34

9 Model predictive control with the learned residual (RQ1) 34
9.1 The MPC problem . . . 34
9.2 Where the learned residual enters . . . 35
9.3 Experiment: steady altitude error under mismatch . . . 35

10 How long may fault detection take? (RQ2, empirical) 36
10.1 The failure timeline . . . 36
10.2 Why the undetected phase is dangerous: a simple bound . . . 37
10.3 Experimental design (E4) . . . 37
10.4 Results . . . 38
10.5 Do the MPCs tolerate more delay? . . . 40

11 A first mathematical certificate (RQ2) 40
11.1 The idea: an energy-like function that always decreases . . . 40
11.2 Handling the thrust limits (Hu and Lin) . . . 41
11.3 Turning it into LMIs . . . 41
11.4 Results . . . 42
11.5 From here to a certified delay budget (planned) . . . 43

12 Discussion, limitations and next steps 44
12.1 Status of the hypotheses . . . 44
12.2 What the results mean . . . 44
12.3 Limitations . . . 44
12.4 Plan for the remaining weeks . . . 45
12.5 Conclusion . . . 45

A Glossary 46

B Symbols 48

C Software and reproducibility 49

1 Introduction

1.1 How a quadrotor flies

A quadrotor has four propellers, each driven by its own electric motor. Each propeller pushes air down and so produces an upward force called *thrust*. A quadrotor has no wings and no rudder: everything it does, it does by changing the four thrusts.

- **Going up and down.** If all four thrusts together are larger than the weight, the drone climbs; if smaller, it descends.
- **Tilting (roll and pitch).** If the propeller on one side pushes harder than the one on the opposite side, the drone tilts. A tilted drone has its thrust pointing partly sideways, so it moves sideways. This is how a quadrotor travels horizontally.
- **Turning (yaw).** A spinning propeller also pushes back on the drone with a small twisting effect (a *reaction torque*), in the direction opposite to its spin. Two of the propellers spin clockwise and two anticlockwise, so in normal flight the twisting effects cancel. To turn, the drone speeds up the pair that spins one way and slows down the other pair.

Four thrusts control exactly four things: total upward force, roll, pitch and yaw. There is no spare actuator. This is why losing even one rotor is serious.

1.2 What happens when a rotor fails

If one rotor stops completely, only three thrusts remain, but the drone still needs to (i) carry its weight, (ii) keep from rolling over and (iii) cancel the yaw twist. With three propellers this is impossible: the reaction torques of the remaining propellers no longer cancel, so the drone starts to spin about its vertical axis, faster and faster, until air drag on the spinning body balances the twist.

Mueller and D'Andrea showed that this is not the end [1, 2]. If we *give up* control of the yaw angle (the compass heading) and accept that the drone spins, the remaining three rotors can still hold the drone in the air and steer it. For their drone the paper predicts a constant spin of about 20 rad/s (about 3 turns per second) around an axis tilted about 17° from the body's vertical axis, while the centre of mass moves on a tiny circle of about 8 mm radius (our simulator gives 19.6 rad/s, 16° and 7 mm; Section 6 explains the small difference). They called this state the *relaxed hover*, and they flew it on a real drone.

In plain words

Think of a spinning top. It cannot stand still upright, but it can stand steadily while spinning. After losing a rotor the drone becomes something like a flying spinning top: it is stable only while it spins, and it moves around by tilting its spin axis.

1.3 The two problems this project studies

Getting a real drone into this spinning state raises two connected problems.

Problem 1: the model is not perfect. A controller is designed using a mathematical model of the drone. The physics model of Mueller and D'Andrea captures the main spinning motion well, but it misses some real effects. For example, in their two-propeller flight the drone stayed about 2.4 m below its set-point, which the authors attribute to less thrust while spinning fast, and in the one-propeller flights the drone spun about 20 to 27% slower than predicted [1, 2]; the inertia and motor speed are never known exactly. A controller built on an imperfect model makes systematic errors, for example it settles at the wrong height. A natural fix is to keep the physics model and *learn its error* from data. We learn the error with the tools of Koopman methods

(EDMD, Section 2): a fixed dictionary of nonlinear features of the state and input, and least-squares regression. In the controller the learned correction is evaluated outside the optimization, as Todde et al. do [3], so the optimization problem stays easy (convex) to solve. Such "residual Koopman" models already exist for healthy drones and other robots [3–5], but we did not find them applied to the spinning state after a complete rotor loss.

Problem 2: detection takes time. The flight computer does not know immediately that a rotor has failed. A fault-detection algorithm has to notice it first, and published detectors need about 20 to 130 ms [6–8]. During this time the normal (healthy-flight) controller keeps flying a drone that has lost a rotor, and the drone tilts and drops. When the failure is finally detected, the failure-mode controller must catch the drone from wherever it has drifted. Several recovery methods that come with a mathematical guarantee start their analysis at the moment the failure is known and do not model the undetected interval [9, 10], and many studies assume the failure is known in advance [8]. For linear systems whose remaining actuators can still compensate a fault, LMI methods exist that maximize the tolerated detection delay under actuator saturation [11, 12]. For complete rotor loss of a quadrotor, where the steady state after the failure is a spin, we did not find a certified *maximum* tolerable detection delay that accounts for the limited thrust of real motors.

1.4 Research questions

RQ1 (as proposed). Can a Koopman-style (EDMD) model of only the mismatch, built on top of the known spinning-equilibrium model of Mueller and D'Andrea, predict and control the failed drone better than the physics model alone, or than a Koopman model of the full dynamics? "Better" means a smaller multi-step prediction error on new data, and a smaller steady error in closed loop.

RQ2 (added after the proposal). What is the largest fault-detection delay $\tau^\star$ for which recovery into the spinning state can be guaranteed mathematically (as a first step, for the linearized model with thrust limits), given the healthy controller, the failure-mode controller, thrust limits, motor lag and sampling? How do the controllers of RQ1 change $\tau^\star$?

The two questions are linked: every controller that catches the drone after detection (LQR, analytical MPC, residual Koopman MPC) can be compared by the detection delay it tolerates.

Hypotheses. Before the experiments we wrote down what we expected:

H1. A Koopman model of the full failed-drone dynamics has eigenvalues outside the unit circle and loses to a "nothing changes" predictor at long horizons.

H2. The residual model has a smaller multi-step prediction error than both the physics model and the full Koopman model.

H3. An MPC that uses the residual removes the steady error that model mismatch causes in an MPC built on the physics model.

H4. The residual model reaches a given prediction error with less training data than the full Koopman model.

H5. A certificate built from the linearization at the spinning equilibrium does not cover the state right after a failure; one valid over the whole spin-up does.

H6. The certified $\tau^\star$ is finite and of the same order as real detector delays.

H7. The certified $\tau^\star$ is within 2 to 3 times the delay found by simulation.

H1 to H4 belong to RQ1 and H5 to H7 to RQ2. Section 12 gives the status of each.

1.5 What we did

Everything is done in simulation; no hardware is used. In order:

1. We wrote a simulator of the drone used by Mueller and D'Andrea (Sections 4 and 5).
2. **V0, E1:** we checked it against three published papers, against flight data and against hand-derived physics results (Section 6).
3. **E2, E3:** we analysed the linearized models (controllability, observability) and designed LQR controllers (Section 7).
4. **E6:** we built three predictors of the failed drone (physics only, full Koopman, residual Koopman) and compared them (Section 8).
5. **E7:** we built MPC controllers on the physics model with and without the learned residual (Section 9).
6. **E4:** we measured how the probability of a safe recovery falls as detection takes longer (Section 10).
7. **E5:** we computed a first mathematical certificate (an LMI-based invariant ellipsoid) and compared it with simulation (Section 11).

The labels V0 and E1 to E7 are the experiment names used in figure titles and in the code (Appendix C).

What is reproduction and what is new. The model, the equilibrium, its controllability, the LQR weights and the spin-up strategy are from Mueller and D'Andrea; we reimplemented and checked them, and found one inconsistency in a published case. New in this work, as far as our search found: the comparison of physics-only, full Koopman and residual models in the spinning regime after complete rotor loss and its use in MPC (RQ1); the idle-thrust argument and the measured probability of safe recovery against detection delay; and the first saturation-aware certificate for this controller, as a step towards a certified delay budget (RQ2). Section 12 discusses the results, the limitations and the remaining work. Appendix A is a glossary of all technical terms, Appendix B lists the symbols, and Appendix C explains how to reproduce every number.

1.6 Main results in one table

Table 1: Main results so far. Details and caveats are in the sections given.

Topic	Result	Section
Simulator validation	Two-rotor and three-rotor-loss equilibria of [1] reproduced to every published digit; five cases of [2] within 4%; flight spin-up time within 4%	6
Linear analysis	Failed drone is controllable from three rotors; gyroscopes alone cannot see tilt or height	7
LQR baseline	Recovers on the nonlinear model; tolerates only 18 ms of input delay (healthy LQR: 40 ms)	7
Prediction (RQ1)	Residual model best from 100 ms to 1 s (1 s error 0.64 vs 0.91 full Koopman vs 7.09 physics only); needs less data	8
Generalization (RQ1)	Learned models fail on a plant with different parameters	8
MPC (RQ1)	Residual Koopman MPC cuts the steady altitude error by 32% on the training plant; on one held-out variant (plant B without its thrust loss) the error is 47% larger	9
Detection delay (RQ2)	Safe-recovery probability below 0.9 from 12 ms and below 0.5 from 19 ms of delay	10
Certificate (RQ2)	For the linearized model with thrust limits: certified 22% of the way to the failure state; simulation recovers up to 70%	11

2 Background: the ideas we use

This section explains, in simple terms, the ideas from control theory, numerical computing and machine learning that the rest of the report uses. A reader who knows them can skip to Section 3. Each idea is revisited with its exact equations where it is used.

2.1 Systems, states, inputs and outputs

A *system* is anything whose behaviour changes over time and can be influenced: here, the drone. Its *state* $\boldsymbol{x}$ is the smallest list of numbers that, together with future inputs, determines its future. For a drone the state contains position, velocity, orientation (attitude), rotation rates and, because motors cannot change speed instantly, the current thrust of each motor. The *inputs* $\boldsymbol{u}$ are what we command: the four thrusts. The *outputs* $\boldsymbol{y}$ are what sensors measure.

A *state-space model* writes the system as a differential equation

$$\dot{\boldsymbol{x}}(t) = f(\boldsymbol{x}(t), \boldsymbol{u}(t)), \qquad \boldsymbol{y}(t) = h(\boldsymbol{x}(t)), \tag{1}$$

where the dot means the rate of change in time. The function f comes from physics (Newton's laws). If f has the special form $A\boldsymbol{x} + B\boldsymbol{u}$ with constant matrices A and B, the model is *linear time-invariant* (LTI). Linear models are much easier to analyse and to control, so a large part of control theory is about them.

2.2 Equilibria and linearization

An *equilibrium* (also called a *trim point*) is a pair $(\bar{\boldsymbol{x}}, \bar{\boldsymbol{u}})$ at which nothing changes: $f(\bar{\boldsymbol{x}}, \bar{\boldsymbol{u}}) = 0$. For a healthy drone, hover with each motor carrying a quarter of the weight is an equilibrium. For a drone with a failed rotor, the relaxed hover is an equilibrium, but only in coordinates that rotate with the drone (Section 4).

Close to an equilibrium, a smooth nonlinear model behaves almost like a linear one. Writing small deviations $\delta\boldsymbol{x} = \boldsymbol{x} - \bar{\boldsymbol{x}}$ and $\delta\boldsymbol{u} = \boldsymbol{u} - \bar{\boldsymbol{u}}$, a first-order Taylor expansion gives

$$\delta\dot{\boldsymbol{x}} \approx A\,\delta\boldsymbol{x} + B\,\delta\boldsymbol{u}, \qquad A = \frac{\partial f}{\partial \boldsymbol{x}}\Big|_{\bar{\boldsymbol{x}},\bar{\boldsymbol{u}}}, \quad B = \frac{\partial f}{\partial \boldsymbol{u}}\Big|_{\bar{\boldsymbol{x}},\bar{\boldsymbol{u}}}. \tag{2}$$

The matrices of partial derivatives A and B are called *Jacobians*, and the process is called *linearization*. We compute the Jacobians numerically by *central differences*: change one entry of $\boldsymbol{x}$ by a tiny amount ϵ up and down and divide the change in f by 2ϵ.

2.3 Eigenvalues and stability

The behaviour of $\dot{\boldsymbol{x}} = A\boldsymbol{x}$ is described by the *eigenvalues* λ of A (numbers with $A\boldsymbol{v} = \lambda\boldsymbol{v}$ for some vector $\boldsymbol{v}$). Each eigenvalue is a *mode*: a pattern of motion that grows or decays like $e^{\lambda t}$.

- Real part negative: the mode decays (stable). The more negative, the faster.
- Real part positive: the mode grows (unstable).
- Real part zero: a simple (non-repeated) eigenvalue gives a mode that neither grows nor decays (marginally stable), for example an undamped oscillation or a single integrator. A repeated zero eigenvalue that forms a chain, such as the double integrator from force to position, grows linearly in time and is therefore unstable.
- Imaginary part $\pm j\omega$: the mode oscillates with angular frequency ω rad/s.

The *damping ratio* $\zeta = -\operatorname{Re}\lambda/|\lambda|$ of an oscillating mode says how quickly the oscillation dies out: $\zeta = 1$ means no overshoot, ζ near 0 means a long ringing.

Discrete time. Computers act at fixed time steps of length T_s (the *sample period*). A discrete-time model is $\boldsymbol{x}_{k+1} = A_d\boldsymbol{x}_k + B_d\boldsymbol{u}_k$, where k counts steps. If the input is held constant between samples (a *zero-order hold*, ZOH), the exact discrete matrices are $A_d = e^{AT_\mathrm{s}}$ and $B_d = \int_0^{T_\mathrm{s}} e^{As}\,ds\,B$. In discrete time a mode is stable if its eigenvalue lies inside the unit circle, $|\lambda| < 1$. The largest $|\lambda|$ is the *spectral radius*; a value above 1 means some prediction errors grow at every step.

2.4 Numerical simulation

The nonlinear equations cannot be solved by hand, so we integrate them step by step. We use the classical fourth-order *Runge–Kutta* method (RK4): in each step of length Δt it evaluates f four times (at the start, twice at the middle, at the end) and takes a weighted average. Its error per unit time shrinks like Δt^4, so halving the step makes the error about 16 times smaller. We use $\Delta t = 0.5$ to 1 ms, much shorter than the fastest motion of the drone.

2.5 Feedback control and LQR

A *feedback controller* measures the state and computes the input from it, for example $\boldsymbol{u} = -K\boldsymbol{x}$ with a *gain matrix* K. The closed loop is then $\dot{\boldsymbol{x}} = (A - BK)\boldsymbol{x}$, and the job of control design is to choose K so that all eigenvalues of $A - BK$ have negative real parts with good damping.

The *linear-quadratic regulator* (LQR) chooses K by minimizing a cost

$$J = \int_0^\infty \left(\boldsymbol{x}^\top Q\boldsymbol{x} + \boldsymbol{u}^\top R\boldsymbol{u}\right) dt, \tag{3}$$

where the weight matrix Q penalizes state errors and R penalizes control effort. Large Q entries make the controller aggressive on those states; large R makes it gentle on the motors. The optimal gain is $K = R^{-1}B^\top P$, where P solves the *algebraic Riccati equation* $A^\top P + PA - PBR^{-1}B^\top P + Q = 0$. A common way to choose the weights is *Bryson's rule*: $Q_{ii} = 1/(\text{largest acceptable error in state } i)^2$ and similarly for R.

2.6 Controllability and observability

A system is *controllable* if the inputs can move the state from anywhere to anywhere. For an LTI system this holds exactly when the *controllability matrix* $[B, AB, A^2B, \ldots, A^{n-1}B]$ has full rank n (the number of states). (The rank is computed from the *singular values* of the matrix, which measure how much it stretches vectors in each direction; a singular value near zero means a direction it almost loses.) The *PBH test* (Popov–Belevitch–Hautus) checks the same property mode by mode: mode λ is controllable if the matrix $[\lambda I - A,\ B]$ has full rank.

A system is *observable* if the state can be worked out from the sensor outputs over time. The test is the same with $(A^\top, C^\top)$ in place of (A, B), where $\boldsymbol{y} = C\boldsymbol{x}$. A rank test only says yes or no. To say *how well* a direction can be seen or reached, we use *Gramians*: the observability Gramian $W_o(T) = \int_0^T e^{A^\top s} C^\top C e^{As} ds$ measures how much output "energy" each initial state produces in time T. A tiny smallest eigenvalue means a direction that is almost invisible to the sensors.

2.7 Model predictive control (MPC)

MPC repeatedly solves a small optimization problem. At every sample it uses a model to predict the next N steps (the *horizon*), chooses the input sequence that minimizes a cost (like the LQR cost) while respecting limits (for example motor thrust limits), applies only the first input, and repeats at the next sample with a new measurement. Re-solving at every step from a new measurement adds feedback: a prediction error made at one step is corrected at the next.

When the model is linear, the cost quadratic and the limits linear, the optimization is a *quadratic program* (QP). With a positive definite weight on the inputs (as in our MPC) the QP is *strictly convex*: it has a single best solution, and solvers find it reliably and fast. Keeping the problem convex is the main reason we want linear prediction models.

2.8 Saturation and delay

Real motors have limits. *Saturation* means that a command outside the range $[f_{\min}, f_{\max}]$ is clipped. A controller that asks for more than the motors can give behaves differently from its linear design, and can even become unstable. *Delay* means that the controller acts on old information. Every feedback loop tolerates only a certain pure delay before it becomes unstable; this amount is its *delay margin*.

2.9 Koopman operators and EDMD

The *Koopman operator* is a way to describe a nonlinear system with linear equations. Instead of following the state $\boldsymbol{x}$ itself, one follows many functions of it, $\psi_1(\boldsymbol{x}), \psi_2(\boldsymbol{x}), \ldots$ (called *observables*, collected into a *dictionary*). For a system without inputs and a well-chosen, possibly infinite, set of observables, their evolution is exactly linear. In practice one uses a finite dictionary (for example all products of pairs of states) and finds the best linear map from data, so the result is an approximation; with inputs, the lifted model below is an approximation even for a rich dictionary [13]. This is *extended dynamic mode decomposition* (EDMD) [14]; its extension to inputs [13] we call EDMDc. The resulting model, $\boldsymbol{z}_{k+1} = A\boldsymbol{z}_k + B\boldsymbol{u}_k$ with $\boldsymbol{z} = \psi(\boldsymbol{x})$, is linear in a bigger *lifted* state $\boldsymbol{z}$, so it can be used directly in a convex MPC.

In plain words

Imagine predicting the motion of a ball on a curved track. Its position alone changes in a complicated, nonlinear way. But if you also keep track of its speed, the square of its speed, its height and so on, the rules that update all these numbers together can be close to a simple linear recipe. Koopman methods look for such a set of numbers and the recipe that updates them.

2.10 Regression, ridge regression and residual learning

Linear regression finds a matrix W so that $\boldsymbol{y} \approx W^\top \boldsymbol{\phi}$ for many examples $(\boldsymbol{\phi}, \boldsymbol{y})$, by minimizing the sum of squared errors. *Ridge regression* adds a small penalty $\lambda\|W\|^2$; this keeps W from becoming huge when some features are nearly duplicates, and reduces *overfitting* (fitting the noise of the training data instead of the true pattern). The solution has the closed form $W = (\Phi^\top\Phi + \lambda I)^{-1}\Phi^\top Y$.

Residual learning keeps a trusted model and learns only what it gets wrong. If the physics model predicts $\boldsymbol{x}_{k+1}^{\text{nom}}$ and the truth is $\boldsymbol{x}_{k+1}$, the learned part predicts the *residual* $\boldsymbol{e}_{k+1} = \boldsymbol{x}_{k+1} - \boldsymbol{x}_{k+1}^{\text{nom}}$. The hope is that the residual is small and simple, so it can be learned from little data.

2.11 Lyapunov functions and linear matrix inequalities

A *Lyapunov function* $V(\boldsymbol{x})$ is like an energy: it is zero at the equilibrium, positive elsewhere, and always decreases along the motion of the system. If such a function exists, the system returns to the equilibrium. A common choice is $V = \boldsymbol{x}^\top P\boldsymbol{x}$ with a *positive definite* matrix P (written $P \succ 0$), which means $\boldsymbol{x}^\top P\boldsymbol{x} > 0$ for every $\boldsymbol{x} \neq 0$; $P \succeq 0$ (positive semidefinite) allows equality, and $P \preceq 0$ means $-P \succeq 0$. The set where $V \leq 1$ is an *ellipsoid*; if V decreases everywhere on it, no trajectory can leave it, so it is an *invariant set*, and every state inside it is guaranteed to recover.

A *linear matrix inequality* (LMI) is a condition of the form "this symmetric matrix, which depends linearly on unknown matrices, is negative (or positive) semidefinite". The conditions for a Lyapunov function can often be written as LMIs. LMIs are convex, so a computer can find the largest certified ellipsoid reliably [15]. Section 11 explains this in detail.

2.12 Statistics used

- **Monte Carlo simulation**: run the same experiment many times from random starting conditions and count outcomes.
- **Wilson interval**: a 95% confidence interval for a success probability estimated from k successes in n trials; it behaves well even when k is 0 or n (Section 10).
- **RMSE**: root mean square error, $\sqrt{\text{mean of squared errors}}$. We divide each state by its typical size before averaging, so that states with different units can be combined (*normalized* RMSE).

3 Related work: what the papers did

This section describes the papers our work builds on, what each one contains, and how we use it. We searched web search engines, Semantic Scholar, OpenAlex, Crossref and arXiv, and followed the citation trails of the key papers (about 620 citing titles screened). We say how deeply we read each paper, because a claim can only be as reliable as the reading behind it. "Full" means the whole text was read; "partial" means the relevant sections; "abstract" means only the abstract or descriptions by citing papers.

3.1 The three papers we validate against

Mueller and D'Andrea, ICRA 2014 [1] (read in full). This is the paper our project rests on. It contains:

- **A model** of a quadrotor (their eqs. 1 to 14) that includes the gyroscopic effect of the spinning propellers (explained in Section 4) and a linear yaw drag torque $-\gamma r$. The propeller's own angular acceleration term is neglected.

- **Vehicle parameters** (their Sec. V-A; the motor time constant is in Sec. V-C), based on an Ascending Technologies Hummingbird (a commercial research quadrotor): mass 0.50 kg, arm length 0.17 m, inertias $I^T_{xx} = 3.2 \times 10^{-3}$ and $I^T_{zz} = 5.5 \times 10^{-3}\,\mathrm{kg\,m^2}$, propeller inertia $1.5 \times 10^{-5}\,\mathrm{kg\,m^2}$ (printed as "kg mm^2" in the paper, an evident typo), thrust coefficient $6.41 \times 10^{-6}\,\mathrm{N\,s^2/rad^2}$, torque-to-thrust ratio 0.0169 m, yaw drag $2.75 \times 10^{-3}\,\mathrm{N\,m\,s/rad}$, thrust range 0.2 to 3.8 N per rotor, and a motor time constant of 15 ms. We use every one of these values.
- **The relaxed-hover equilibria** for the loss of one propeller (their eqs. 50 to 53: thrusts (2.05, 1.02, 2.05, 0) N, body rates (0, 5.69, 18.89) rad/s, axis (0, 0.289, 0.958), orbit radius 8 mm for the thrust ratio 0.5), of two opposing propellers (eqs. 57 to 59) and of three propellers (eqs. 62 to 64), plus a figure (Fig. 3) of how the one-rotor equilibrium changes with the thrust ratio.
- **A linear model** of the reduced attitude about the equilibrium, the statement that it is controllable (for one lost propeller) when $l \neq 0$ and $\bar{n}_z \neq 0$, and the coupling constant $\bar{a}$ (their eq. 28) that decides when controllability is lost (eqs. 37 to 39).
- **An LQR controller** for one lost propeller with weights 1 on the rates, 20 on the reduced attitude, 0 on the motor states and $1\,\mathrm{N^{-2}}$ on the inputs, with motor dynamics included as extra states (the two-propeller design uses different weights).
- **A flight experiment** (their Fig. 4): propeller 4 is switched off at $t = 0$ *with the failure known*. The drone first spins up on the two opposing propellers 1 and 3, and the LQR is switched on once the rotation rate exceeds 10 rad/s, which took 0.62 s. Reading the plot, during spin-up rotors 1 and 3 run at a constant thrust of about 2.95 N with rotor 2 stopped; the drone then climbs from 2.0 to about 2.9 m before settling a few tenths of a metre from its set-point (the paper does not discuss this offset). The measured spin in hover was $\boldsymbol{\omega} = (2.1, 3.7, 13.7)$ rad/s (their eq. 55), about 14.3 rad/s against about 19.7 rad/s predicted. In a separate two-propeller flight (their Fig. 5) the drone stayed about 2.4 m below its set-point, which the authors attribute to the propellers producing "significantly less thrust than expected when the body is rotating at high angular velocities".
- **Future work** stated by the authors: "implement the presented results with a fault detection scheme, and investigate different switching strategies for transitioning from e.g. four propellers to three". This is exactly where RQ2 sits.

Our use: the model, all parameters, the equilibria and Fig. 3 as validation targets, the LQR weights, the spin-up strategy and the flight spin-up time as a flight-data check; the thrust loss seen in the two-propeller flight motivates our truth plants (Section 4.10).

Mueller and D'Andrea, IJRR 2016 [2] (Sections 1 to 5 read). A more general and more detailed version for any multicopter. Two physical effects are added: (i) the thrust of a propeller depends on its speed relative to the *air*, $\Omega_i + r$, not relative to the body, because the whole body spins at rate r; (ii) the drag torque is *quadratic* in the body rate, $-\|\boldsymbol{\omega}\| K_d \boldsymbol{\omega}$, on all three axes. The paper expresses its hover solutions in propeller speeds and lists, in its Sec. 5.2 (eqs. 31 to 50), the hover solutions (propeller speeds and body rates) for five cases: no failure, one rotor lost, two opposing rotors lost, two adjacent rotors lost and three rotors lost. In flight with one rotor lost, the measured spin rate was 15.8 rad/s against 19.9 rad/s in simulation (Sec. 5.4); the authors suggest additional torques from the translation of the propeller centres as a possible cause. *Our use:* we wrote an independent implementation of this model and checked it against the five published solutions.

Sun, Wang, Chu and de Visser, T-RO 2021 [16] (abstract and relevant equations read). An incremental nonlinear dynamic inversion (INDI) controller for a Parrot Bebop 2

that loses two opposing rotors, tested in a wind tunnel at speeds above 8 m/s. An LQR of the Mueller–D'Andrea type is compared and degrades in these conditions. The paper gives a closed-form equilibrium yaw rate for two lost rotors (its eq. 60) and the Bebop 2 parameters (its Table I in the arXiv version, which the text refers to as Table III). *Our use:* a second, different airframe to test our equilibrium solver.

3.2 Other controllers for complete rotor loss

- **Freddi, Lanzon and Longhi** [17, 18] (abstract): give up yaw and stabilise a constant-rate spin about the vertical axis with level attitude, using feedback linearization and H_∞ loop shaping (a robust linear design method) around this working point.
- **Stephan, Schmitt and Fichter** [19] (abstract only): an LPV (linear parameter-varying) controller in which the yaw rate is a time-varying scheduling parameter, with guaranteed performance during the transient after the actuator loss. We have not been able to read the full text, so we do not know whether thrust saturation is treated. This is the closest work to our planned certificate over the spin-up.
- **Khattab, Alwi and Edwards** [20] (abstract): an LPV reduced-attitude model with sliding-mode control.
- **Sun, Baert, Strack van Schijndel and de Visser** [21] (partial): recovery from large upsets using a quadratic-programming control allocation and the "attainable moment set". Tested by Monte Carlo and by throwing the drone by hand. It states that a drone "may be upside down and fast rotating before the FTC is triggered, because of ... delay of the fault detection module", and that linear controllers "could fail" through linearization errors and saturation. This sentence directly motivates RQ2, but the paper does not quantify the delay.
- **Nan, Sun, Foehn and Scaramuzza** [22] (partial): nonlinear MPC with per-rotor thrust limits that recovers even from a failure at the top of a flip, in simulation and flight. The authors state that fault detection is outside the scope of the paper; the failed rotor is reported instantly by a mock detector, so there is no fault-detection-delay analysis.
- **Sun, Zhang and Lang** [23] (abstract): nonlinear MPC that learns an unknown complete rotor failure online.
- **Ke, Cai and Quan** [24] (abstract): a *passive* controller that tolerates one to three rotor failures without any detection step and without switching.

3.3 Guarantees and detection delay

- **Simha, Vadgama and Raha** [9] (full): a geometric controller with an analytic region of attraction (a set of starting states that is proven to recover). The fault is assumed to be known from $t = 0$; positive thrust is only checked in simulation; no motor lag.
- **Al Lawati and Akhtar** [10] (partial, assumptions checked in the text): quasi-static feedback with path invariance and local exponential convergence. They write that they "assume the availability of a reliable FDI module and an instantaneous transition".
- **Başak and Prempain** [25] and **Başak's thesis** [26] (conference abstract and thesis Ch. 6 read): the closest prior work to RQ2. The thesis treats *intermittent* faults, in which a motor (or the control of an actuator) is lost and regained, as a switched linear system (a system that jumps between several linear models). The undetected interval is modelled as "faulty plant with the healthy controller" for a fixed time, $T_D = 100$ ms for the motor-loss case and 500 ms for loss of control; LMI conditions with several Lyapunov functions guarantee stability as long as switching is not too frequent (an average "dwell time" of about 1.1 s).

The delay is fixed in advance rather than maximized, thrust saturation is not included, and the linearizations use Euler angles and the yaw rate instead of the relaxed-hover model.

- **Qi, Theilliol, He and Han** [27] (abstract): a "recoverable set" (an invariant set of states that can be driven to the equilibrium) at the relaxed hover, with rotor-speed limits, computed by sum-of-squares programming (an extension of LMI methods to polynomial functions). It also covers partial loss of effectiveness and the loss of more than one propeller. Whether it covers the spin-up is unknown to us.
- **Measured tolerances** are empirical: 180 ms for a variable-pitch "heli-quad", found by simulation sweeps [28] (partial read; a different vehicle that keeps yaw control). Other papers report the detection delays their controllers worked with, not a maximum: in simulation, nonlinear MPC and INDI recovered with a detection delay of about 0.1 s [29], and a nonlinear MPC recovered in hover with 0.21 to 0.22 s, while success fell to 60 to 90% at 0.18 to 0.25 s during trajectory tracking [8] (full texts read).
- **Detector speed**: Strack van Schijndel, Sun and de Visser [6] report detection of propeller loss in 30 to 130 ms (26 propeller ejections on a Bebop 2, 95% interval 28 to 132 ms); Yang et al. [7] report about 45 ms, and Zhou et al. [8] detect a stopped motor from its measured speed in about 20 to 30 ms. So published detectors need about 20 to 130 ms, depending on the method and the sensors.
- **General theory**: set-theoretic fault-tolerant control uses invariant and reachable sets for detection and reconfiguration [30]; Shin and Belcastro [31] compute, with LMIs, a bound on the performance of a fault-tolerant system as a function of the detection time; Rotondo et al. [11, 12] design controllers that maximize the tolerated delay between a fault and its isolation, with actuator saturation, for linear and LPV systems whose remaining actuators can still compensate the fault; a motion planner based on backward reachability exists for the failed quadrotor after the fault is handled [32].

3.4 Koopman models and residual learning

- **EDMD and EDMDc** (partial): Williams, Kevrekidis and Rowley [14] introduced EDMD, which fits a linear operator on a dictionary of observables by least squares. Korda and Mezić [13] extended EDMD to systems with inputs and showed how the resulting linear predictor is used in MPC; we call this EDMDc ("with control"). With inputs, the lifted linear model is an approximation even for a rich dictionary. Brunton et al. [33] discuss which observables make the dynamics linear.
- **Koopman MPC for healthy quadrotors** (abstracts): SE(3) Koopman MPC [34] and analytical Koopman lifting [35].
- **Koopman with faults** (abstracts): Lee et al. [36, 37] address loss-of-effectiveness actuator faults (and, for detection, bias faults), not complete rotor loss, with Koopman MPC and Koopman fault detection. The MPC paper updates the input matrix online with a weighted-window EDMD.
- **Physics plus learned Koopman residual**: soft robot arms [4] (abstract), ground vehicles [38] (abstract), quadrupeds [5] (full text; it compares a nominal model, full-state Koopman models and a residual Koopman model, as we do, and reports that a full-state SE(3) Koopman MPC baseline becomes unstable within about 5 s in simulation), and healthy quadrotors with a deep Koopman lifting that is updated online [39] (full text). Bruder et al. and Fu et al. also report that residual models need less training data than purely data-driven ones.
- **Todde, Martini, Rizzo and Valavanis, ICUAS 2026** [3] (read in full): the closest Koopman work. A small neural network (8 inputs, layers of 64 and 32, 4 outputs,

2,788 parameters) predicts the one-step error of an analytical Koopman model of a *healthy* quadrotor. The prediction is evaluated *outside* the MPC optimization, injected as a measured disturbance (so the QP stays convex) and clipped by $d_{\max}$ taken from percentiles of the training errors. Training data: 20 runs of 80 s with mass, inertia, arm length and motor time constant randomized by ±20%. In a single simulated stress test with +30% mass, inertia and arm length and −10% thrust and torque coefficients (outside the training range), the overall position RMSE drops from 0.146 m (Koopman MPC) to 0.095 m (residual Koopman MPC); under nominal conditions the residual version is worse (0.040 against 0.017 m). Simulation only, no rotor failure, no guarantees; formal robustness guarantees are listed as future work. *Our use:* we borrow three ideas: residual outside the QP, clipping, and randomized-parameter training data.

- **Koopman with guarantees** (abstract): SafEDMD [40] designs controllers by semidefinite programming with bounds on the EDMD error, which is the starting point for a future certificate of the residual Koopman loop.

3.5 Tools

Hu and Lin [41] give LMI conditions for invariant ellipsoids of linear systems with saturating inputs, which we use in Section 11. Boyd et al. [15] is the standard reference for LMIs in control. Practical attitude estimators for the spinning drone use inertial sensors together with vision [42] or a magnetometer [43].

3.6 The gap

1. We did not find residual Koopman models applied to the complete-rotor-loss, spinning-equilibrium regime (RQ1). The *pattern* of physics plus learned residual exists for healthy drones [3, 39], and a learned failure model inside MPC exists for complete loss [23]; the combination with the relaxed-hover model is what is new.
2. Several guaranteed recovery methods start their analysis at the moment the failure is known [9, 10]; passive schemes avoid detection altogether [24]; Başak handles a fixed delay without saturation [26]; and maximum tolerated delays under saturation have been certified for linear systems whose actuators can still compensate the fault [11, 12]. For complete rotor loss, where the steady state after the failure is a spin, we did not find a certified *maximum* tolerable delay that accounts for thrust saturation and the drift of the undetected phase (RQ2).

We state both as the outcome of our search, not as proof that no such work exists.

4 The mathematical model

This section builds the model step by step: frames and layout, forces, torques, motors, attitude, the reduced attitude, the spinning equilibrium, and finally the linear models used for control design. We follow Mueller and D'Andrea [1] throughout and say where we add something.

4.1 Frames, layout and sign conventions

We use two coordinate frames (Figure 1):

- The **inertial frame** is fixed to the ground. Its z axis points *up*, so gravity is $-g\boldsymbol{e}_3$ with $g = 9.81\,\mathrm{m/s^2}$ and $\boldsymbol{e}_3 = (0, 0, 1)$.
- The **body frame** is fixed to the drone and moves with it. Its z axis points along the thrust direction ("up" for the drone). Its x and y axes point along the arms.

The rotation matrix R turns a vector written in body coordinates into the same vector in inertial coordinates. Rotor 1 sits on the $+x$ arm, rotor 2 on $+y$, rotor 3 on $-x$ and rotor 4 on $-y$, each at distance l from the centre. Rotors 1 and 3 spin one way (we give them negative speed), rotors 2 and 4 the other way (positive speed). **Rotor 4 is the failed rotor** throughout, as in the paper; by symmetry any rotor would give the same results.

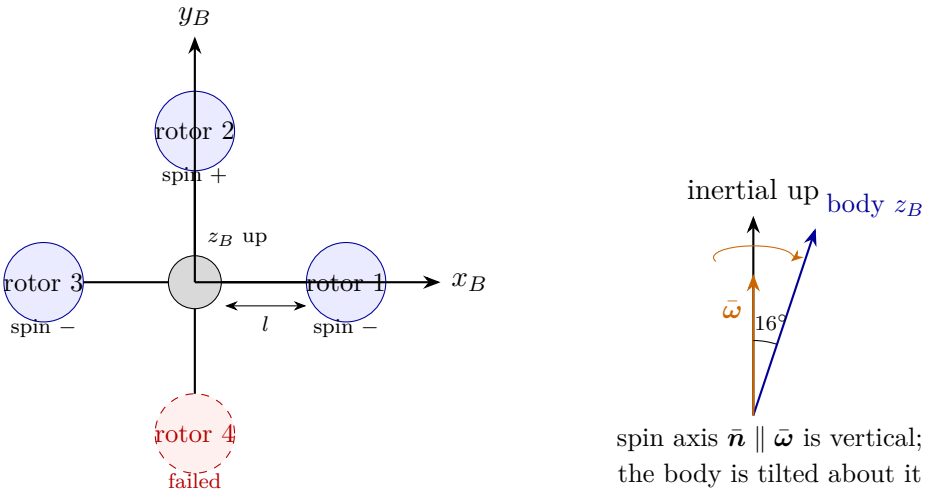


Figure 1: Left: rotor layout seen from above (body frame). Rotors 1 and 3 spin with negative speed, rotor 2 with positive speed; rotor 4 has failed. Right: in the relaxed hover the drone spins about the vertical; its own z axis is tilted by about 16° from the spin axis.

4.2 The state

The simulator state has 17 numbers:

$$\boldsymbol{x} = (\underbrace{\boldsymbol{d}}_{\text{position, 3}}, \underbrace{\boldsymbol{v}}_{\text{velocity, 3}}, \underbrace{\boldsymbol{q}}_{\text{attitude quaternion, 4}}, \underbrace{\boldsymbol{\omega} = (p, q, r)}_{\text{body rates, 3}}, \underbrace{\boldsymbol{f}}_{\text{motor thrusts, 4}}). \tag{4}$$

Position and velocity are in the inertial frame. The body rates p, q, r are the rotation rates about the body x, y and z axes (roll rate, pitch rate and yaw rate), measured in rad/s. The motor thrusts are states because a motor cannot change its thrust instantly.

4.3 Thrust, propeller speed and the motors

Propeller i spinning at speed Ω_i (rad/s; we use Ω for propeller speeds to keep $\boldsymbol{\omega}$ for the body rates) produces thrust $f_i = \kappa_f \Omega_i^2$ along the body z axis and a reaction torque of size $\kappa_\tau f_i$ about the same axis, with sign $(-1)^{i+1}$: rotors 1 and 3 push the body one way, rotors 2 and 4 the other way. From a thrust we recover the signed propeller speed as $\Omega_i = s_i \sqrt{f_i/\kappa_f}$ with signs $s = (-1, +1, -1, +1)$.

Motor lag. Each motor follows its thrust command u_i with a first-order lag of time constant $\sigma = 15\,\text{ms}$:

$$\dot{f}_i = \frac{\text{sat}(u_i) - f_i}{\sigma}. \tag{5}$$

In plain words

A first-order lag means that after a step change of the command, the thrust covers 63% of the change in one time constant (15 ms), 95% in three time constants, and so on, like a capacitor charging.

Saturation. A running motor produces between $f_{\min} = 0.2\,\mathrm{N}$ (idle) and $f_{\max} = 3.8\,\mathrm{N}$. Following the paper, a command $u_i \le 0$ *stops* the motor ($f_i = 0$); this is how the spin-up strategy runs on only two rotors. A failed rotor always produces zero:

$$\mathrm{sat}(u_i) = \begin{cases} 0 & u_i \le 0 \text{ or rotor } i \text{ failed}, \\ \min(\max(u_i, f_{\min}), f_{\max}) & \text{otherwise}. \end{cases} \tag{6}$$

Thrust loss while spinning (test plants only). In the two-propeller flight of [1] the drone stayed about 2.4 m below its set-point, which the authors attribute to the propellers producing less thrust than expected at high body rates. Our *truth plants* (Section 4.10) multiply every thrust by an efficiency $\eta(\boldsymbol{\omega}) = \max(0, 1 - k\|\boldsymbol{\omega}\|^2)$. The nominal model, used for all designs, has $k = 0$ and $\eta = 1$.

4.4 Translation: Newton's second law

The thrust of every rotor points along the body z axis, which in inertial coordinates is $R\boldsymbol{e}_3$. So

$$m\ddot{\boldsymbol{d}} = R\,\boldsymbol{e}_3\,\eta(\boldsymbol{\omega})\,f_\Sigma - mg\,\boldsymbol{e}_3, \qquad f_\Sigma = \textstyle\sum_{i=1}^{4} f_i. \tag{7}$$

We neglect air drag on the body for translation, as the paper does.

4.5 Rotation: Euler's equation with propellers

The rotational dynamics are the paper's eq. (4) with the propeller's own angular acceleration neglected:

$$I^B\dot{\boldsymbol{\omega}} = \boldsymbol{\tau} - \boldsymbol{\omega} \times (I^T\boldsymbol{\omega} + I^P_{zz}\,\Omega_\Sigma\,\boldsymbol{e}_3), \qquad \Omega_\Sigma = \textstyle\sum_i \Omega_i. \tag{8}$$

Each term has a physical meaning:

- I^B is the inertia matrix of the body without propellers, and $I^T = I^B + 4I^P$ includes the propellers. Both are diagonal, with $I_{xx} = I_{yy}$ by symmetry. As in the paper, the propeller inertia about x and y is neglected, so $I^B_{xx} = I^T_{xx} = 3.2 \times 10^{-3}\,\mathrm{kg\,m^2}$ and $I^B_{zz} = I^T_{zz} - 4I^P_{zz} = 5.44 \times 10^{-3}\,\mathrm{kg\,m^2}$.
- $I^T\boldsymbol{\omega}$ is the angular momentum of the drone rotating as a rigid object.
- $I^P_{zz}\Omega_\Sigma\boldsymbol{e}_3$ is the extra angular momentum of the propellers spinning relative to the body. In normal hover the propellers cancel pairwise ($\Omega_\Sigma = 0$); after a failure they do not.
- $\boldsymbol{\omega} \times (\cdot)$ is the *gyroscopic* term. When a body that carries angular momentum $\boldsymbol{h}$ rotates with rate $\boldsymbol{\omega}$, the direction of $\boldsymbol{h}$ changes, and that needs a torque $\boldsymbol{\omega} \times \boldsymbol{h}$. This is the same effect that makes a spinning top precess instead of falling.

The torque vector from the rotors and the yaw drag is

$$\boldsymbol{\tau} = \begin{pmatrix} l\,(f_2 - f_4) \\ l\,(f_3 - f_1) \\ \kappa_\tau\,(f_1 - f_2 + f_3 - f_4) - \gamma r \end{pmatrix}. \tag{9}$$

The first two rows are lever arm times force: for example rotor 2 at $(0, l, 0)$ pushing up with f_2 gives $(0, l, 0) \times (0, 0, f_2) = (lf_2, 0, 0)$. The third row is the sum of reaction torques minus an air-drag torque that grows linearly with the yaw rate r. In the truth plants every f_i in (9) is multiplied by $\eta(\boldsymbol{\omega})$.

4.6 Attitude

We store the attitude as a *unit quaternion* $\boldsymbol{q} = (q_w, q_x, q_y, q_z)$, four numbers with $\|\boldsymbol{q}\| = 1$ that encode a rotation. Quaternions avoid the "gimbal lock" problem of Euler angles, which matters because the failed drone spins through every heading. The quaternion changes as

$$\dot{\boldsymbol{q}} = \tfrac{1}{2}\, \boldsymbol{q} \otimes (0, \boldsymbol{\omega}), \tag{10}$$

where $\otimes$ is quaternion multiplication. After each integration step the quaternion is divided by its length, so that it stays a unit quaternion despite rounding errors. The rotation matrix $R(\boldsymbol{q})$ is computed from it by the standard formula.

4.7 The reduced attitude

After the failure we give up the heading (the yaw angle), so we do not need the full attitude. We need only the direction of "up" as seen from the drone:

$$\boldsymbol{n} = R^{\top} \boldsymbol{e}_3. \tag{11}$$

If the drone is level, $\boldsymbol{n} = (0, 0, 1)$; if it is upside down, $n_z < 0$. Its rate of change follows from $\dot{R} = R[\boldsymbol{\omega}]_{\times}$ (where $[\boldsymbol{\omega}]_{\times}\boldsymbol{a} = \boldsymbol{\omega} \times \boldsymbol{a}$):

$$\dot{\boldsymbol{n}} = \dot{R}^{\top} \boldsymbol{e}_3 = -[\boldsymbol{\omega}]_{\times} R^{\top} \boldsymbol{e}_3 = -\boldsymbol{\omega} \times \boldsymbol{n}. \tag{12}$$

The vertical acceleration also needs only $\boldsymbol{n}$, because the vertical component of $R\boldsymbol{e}_3$ is $\boldsymbol{e}_3^{\top} R \boldsymbol{e}_3 = n_z$:

$$\ddot{z} = \frac{\eta f_{\Sigma}\, n_z}{m} - g. \tag{13}$$

Equations (8), (12) and (13) together form a closed set in $(\boldsymbol{\omega}, \boldsymbol{n}, z, v_z)$. They do not contain the heading or the horizontal position. This is the key to the whole approach.

4.8 The relaxed hover (spinning equilibrium)

Conditions. We look for a state in which $\boldsymbol{\omega}$ and $\boldsymbol{n}$ are constant in body coordinates and the drone neither climbs nor sinks on average:

1. $\dot{\boldsymbol{\omega}} = 0$ in (8): the torques balance the gyroscopic term.
2. $\dot{\boldsymbol{n}} = 0$ in (12): this needs $\boldsymbol{\omega} \times \boldsymbol{n} = 0$, so the spin axis is parallel to "up": $\bar{\boldsymbol{n}} = \bar{\boldsymbol{\omega}}/\|\bar{\boldsymbol{\omega}}\|$.
3. $\ddot{z} = 0$ in (13): $f_{\Sigma}\, \bar{n}_z = mg$.

Condition 2 says the drone spins about the vertical. The body z axis is tilted from the spin axis, so the thrust has a horizontal part of size $g \tan\theta$ per unit mass (θ is the tilt), which rotates with the spin. The centre of mass therefore moves on a small horizontal circle of radius

$$\rho_{\text{orbit}} = \frac{\sqrt{1 - \bar{n}_z^2}}{\bar{n}_z}\, \frac{g}{\|\bar{\boldsymbol{\omega}}\|^2}. \tag{14}$$

Solving it with rotor 4 failed. With three rotors there are more unknowns than equations, so one free choice is made. Following the paper we set $f_1 = f_3$ and $f_2 = \rho f_1$, with the design ratio $\rho = 0.5$. Then the rate equations can be solved by hand. Write $h_z = I_{zz}^{T} r + I_{zz}^{P} \Omega_{\Sigma}$. With $f_1 = f_3$ and $f_4 = 0$, the torque (9) is $(l f_2,\, 0,\, \kappa_{\tau}(f_1 - f_2 + f_3) - \gamma r)$ and the gyroscopic term is

$$\boldsymbol{\omega} \times (I^{T}\boldsymbol{\omega} + I_{zz}^{P}\Omega_{\Sigma}\boldsymbol{e}_3) = \Big(q[(I_{zz}^{T} - I_{xx}^{T})r + I_{zz}^{P}\Omega_{\Sigma}],\; -p[(I_{zz}^{T} - I_{xx}^{T})r + I_{zz}^{P}\Omega_{\Sigma}],\; 0\Big). \tag{15}$$

Setting $\dot{\boldsymbol{\omega}} = 0$ row by row:

$$\text{row 3:} \quad r = \kappa_\tau \, (f_1 - f_2 + f_3)/\gamma, \tag{16}$$

$$\text{row 2:} \quad p = 0, \tag{17}$$

$$\text{row 1:} \quad q = \frac{l f_2}{(I^T_{zz} - I^T_{xx})\, r + I^P_{zz}\, \Omega_\Sigma}. \tag{18}$$

So for any guess of f_1 we know all thrusts, all propeller speeds, and the rates; then $\bar{\boldsymbol{n}} = \boldsymbol{\omega}/\|\boldsymbol{\omega}\|$, and condition 3 becomes one equation in one unknown, $c(f_1) = f_\Sigma(f_1)\, n_z(f_1) - mg = 0$. We solve it with Brent's method, a safe root finder that needs only an interval where c changes sign. The lower end of the interval is $mg/(2+\rho)$ (the thrust needed if the drone were level, $n_z = 1$).

Result for $\rho = 0.5$.

Quantity	Value	Meaning
$\bar{\boldsymbol{f}}$	(2.040, 1.020, 2.040, 0) N	rotors 1 and 3 work hardest; total 5.10 N
$\bar{\boldsymbol{\omega}}$	(0, 5.37, 18.81) rad/s, $\|\bar{\boldsymbol{\omega}}\| = 19.56$	about 3.1 turns per second
$\bar{\boldsymbol{n}}$	(0, 0.274, 0.962)	body z tilted 15.9° from vertical
propeller speeds	$(-564, +399, -564, 0)$ rad/s, $\Omega_\Sigma = -729$	propellers no longer cancel
orbit radius	7.3 mm	centre of mass nearly still

For comparison, healthy hover needs $mg/4 = 1.23$ N per rotor.

Other failure cases. With rotors 2 and 4 lost and $f_1 = f_3 = mg/2$, the drone spins about its own z axis without tilt, at $r = 2\kappa_\tau f_1/\gamma = 30.1$ rad/s. With only rotor 1 left, the thrust and all three rates must be solved together numerically; the result is $f_1 = 5.37$ N, $\boldsymbol{\omega} = (14.68, 0, 32.99)$ rad/s, $\boldsymbol{n} = (0.407, 0, 0.914)$. We use both cases for validation (Section 6).

4.9 Linear models

Control design uses linear models obtained by linearizing the nonlinear equations at an equilibrium. We build two.

Healthy hover (12 states, 4 inputs). States $(x, y, z, v_x, v_y, v_z, \phi, \theta, \psi, p, q, r)$, where ϕ, θ, ψ are roll, pitch and yaw angles (Euler angles in the order yaw, pitch, roll); inputs are the four thrusts, trimmed at $mg/4$. Euler angles are fine here because the healthy drone stays near level. The open-loop eigenvalues are eleven zeros and $-0.506\,\mathrm{s}^{-1}$ (the yaw-drag mode, $-\gamma/I^B_{zz}$). Near hover, positions, velocities, angles and the roll and pitch rates all act as integrators; the repeated zeros form chains of integrators, so the open loop is unstable.

Failed drone (7 states, 3 inputs). States $\boldsymbol{s} = (p, q, r, n_x, n_y, z, v_z)$ and inputs (f_1, f_2, f_3). Only n_x and n_y are states, because $n_z = \sqrt{1 - n_x^2 - n_y^2}$ follows from them. Because equations (8), (12) and (13) do not depend on the heading or on time, **the linearization about the relaxed hover is time invariant**, even though the drone is spinning in the inertial frame. (In inertial coordinates the spinning equilibrium would be a periodic orbit, and one would need Floquet theory, the stability theory of linear systems with periodic coefficients.) The numerical Jacobians

at $\rho = 0.5$ are

$$A = \begin{pmatrix} 0 & -10.10 & -3.86 & 0 & 0 & 0 & 0 \\ 10.10 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & -0.51 & 0 & 0 & 0 & 0 \\ 0 & -0.96 & 0.27 & 0 & 20.34 & 0 & 0 \\ 0.96 & 0 & 0 & -18.81 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & -2.91 & 0 & 0 \end{pmatrix}, \qquad B = \begin{pmatrix} 3.48 & 48.21 & 3.48 \\ -53.13 & 0 & 53.13 \\ 3.11 & -3.11 & 3.11 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 1.92 & 1.92 & 1.92 \end{pmatrix}. \tag{19}$$

Every entry can be traced to the physics:

- $A_{pq} = -A_{qp} = \bar{a} = \frac{I^T_{xx} - I^T_{zz}}{I^B_{xx}} \bar{r} - \frac{I^P_{zz}}{I^B_{xx}} \bar{\Omega}_\Sigma = -10.10$: the gyroscopic coupling between roll and pitch rate (the paper's eq. 28).
- $A_{pr} = -\bar{q}(I^T_{zz} - I^T_{xx})/I^B_{xx} = -3.86$: a change of yaw rate changes the gyroscopic roll torque.
- $A_{rr} = -\gamma / I^B_{zz} = -0.51$: yaw drag.
- The $\boldsymbol{n}$ rows are the linearization of $\dot{\boldsymbol{n}} = -\boldsymbol{\omega} \times \boldsymbol{n}$. The coefficient of n_y in $\dot{n}_x$ is $\bar{r} + \bar{q}\bar{n}_y/\bar{n}_z = 20.34$, because n_z changes with n_y. The paper's simplified matrix (its eq. 27) uses $\bar{r} = 18.81$ there; we use the exact Jacobian.
- $A_{v_z n_y} = -\bar{f}_\Sigma \bar{n}_y / (m\bar{n}_z) = -2.91$: tilting the thrust away from vertical reduces the vertical force. This entry exists only because the equilibrium is tilted.
- $B_{p,f_2} = l/I^B_{xx} = 53.13$ minus a gyroscopic correction (changing f_2 changes propeller 2's speed and so Ω_Σ), giving 48.21. The same correction gives the small entries 3.48 for f_1 and f_3.
- $B_{v_z,f_i} = \bar{n}_z/m = 1.92$: more thrust, more vertical acceleration.

The eigenvalues of A are $\pm 10.10j$ (the p–q gyroscopic oscillation), $\pm 19.56j$ (the tilt of the spin axis seen from the spinning body; its frequency equals $\|\bar{\boldsymbol{\omega}}\|$), -0.51 (yaw drag) and a double zero (altitude: height is the integral of vertical speed, which is the integral of the vertical force). No eigenvalue has a positive real part, but the open loop is not stable: the two oscillations are undamped and the altitude double integrator drifts (its height error grows linearly). Feedback is needed.

Adding the motors. The motors are added as three more states with $\dot{\boldsymbol{f}} = (\boldsymbol{u} - \boldsymbol{f})/\sigma$; the thrusts in B then act through these states. This gives the 10-state model used by the failure LQR, the MPC and the certificate.

4.10 Truth plants for learning

If we generated learning data with the same model that the controller uses, a learned correction would have nothing to learn, and any improvement would be an artefact. We therefore generate data from *truth plants* that contain effects the nominal model lacks (Table 2). Family A is used for training, family B (different magnitudes, in different directions) only for testing. A third, randomized family draws new values for every training episode; its ranges cover A but not B.

At the relaxed hover ($\|\boldsymbol{\omega}\| \approx 19.6\,\text{rad/s}$), family A loses about 6% of its thrust and family B about 10%.

5 How the model is coded

All code is our own and written in Python 3.12 with NumPy [44] and SciPy [45]. It lives in a small package, `rotorfail/`, plus one script per experiment in `experiments/` and automated

Table 2: Nominal parameters [1] and truth-plant families. k is the thrust-loss coefficient in $\eta = 1 - k\|\boldsymbol{\omega}\|^2$.

Quantity	Symbol	Value
Mass, arm length	m, l	$0.50\,\mathrm{kg}$, $0.17\,\mathrm{m}$
Total inertia	I_{xx}^T, I_{zz}^T	3.2, $5.5 \times 10^{-3}\,\mathrm{kg\,m^2}$
Propeller inertia	I_{zz}^P	$1.5 \times 10^{-5}\,\mathrm{kg\,m^2}$
Thrust coefficient	κ_f	$6.41 \times 10^{-6}\,\mathrm{N\,s^2/rad^2}$
Torque-to-thrust ratio	κ_τ	$0.0169\,\mathrm{m}$
Yaw drag	γ	$2.75 \times 10^{-3}\,\mathrm{N\,m\,s/rad}$
Thrust range, motor lag	$[f_{\min}, f_{\max}]$, σ	$[0.2, 3.8]\,\mathrm{N}$, $15\,\mathrm{ms}$
Family A (training)		$k = 1.5 \times 10^{-4}\,\mathrm{s^2/rad^2}$, $I_{xx} \times 1.08$, $I_{zz} \times 0.95$, $\sigma = 20\,\mathrm{ms}$
Family B (held out)		$k = 2.5 \times 10^{-4}\,\mathrm{s^2/rad^2}$, $I_{xx} \times 0.94$, $I_{zz} \times 1.06$, $\sigma = 12\,\mathrm{ms}$
Randomized		$k \in [0, 2 \times 10^{-4}]$, $I_{xx} \times [0.96, 1.10]$, $I_{zz} \times [0.93, 1.03]$, $\sigma \in [15, 22]\,\mathrm{ms}$

tests in `tests/`. This section walks through the parts that implement Section 4. The listings are excerpts of the real code, shortened for reading (options such as the motor-lag switch, solver fallbacks and iteration limits are left out).

5.1 Package layout

Module	What it contains
`params.py`	All vehicle parameters in one frozen data class `QuadParams`; the truth-plant families
`dynamics.py`	The 17-entry state layout, quaternion helpers, saturation, the equations of motion, one RK4 step
`sim.py`	The simulation loop with a sampled (zero-order-hold) controller
`equilibrium.py`	Solvers for the relaxed hover (one, two and three rotors lost)
`models.py`	The healthy and failed nonlinear models in design coordinates; numerical Jacobians; motor-lag augmentation
`analysis.py`	Controllability and observability tests, Gramians, delay margin
`control.py`	Healthy LQR, failure LQR, spin-up strategy, detection-delay supervisor
`metrics.py`	The success criterion, fixed before the experiments
`koopman.py`	Data generation, dictionary, full-state and residual EDMDc, error evaluation
`mpc.py`	Linear MPC with an optional learned residual
`certify.py`	LMI certificate (saturation-aware invariant ellipsoid)
`validation.py`	Independent implementation of the IJRR 2016 model, used only for validation

5.2 Parameters

`QuadParams` is a frozen data class: once created, its values cannot change by accident. Derived quantities are computed properties, for example $I_{xx}^B = I_{xx}^T - 4I_{xx}^P$ and the torque vector (9):

```
@dataclass(frozen=True)
class QuadParams:
    m: float = 0.50              # kg
    l: float = 0.17              # m, arm length
    Ixx_T: float = 3.2e-3        # total inertia about x and y
    ...
    thrust_loss: float = 0.0     # truth-plant effect k (0 in the nominal model)
    prop_sign: tuple = (-1.0, 1.0, -1.0, 1.0)

    def body_torque(self, f, r):
        f1, f2, f3, f4 = f
        return np.array([self.l * (f2 - f4),
```

```
                    self.l * (f3 - f1),
                    self.kappa_tau * (f1 - f2 + f3 - f4) - self.gamma * r])
```

A truth plant is created with `dataclasses.replace`, which copies the nominal parameters and changes only some of them, for example family A: `replace(base, thrust_loss=1.5e-4, Ixx_T=base.Ixx_T*1.08, ...)`.

5.3 Equations of motion

`deriv(x, f_cmd, p, alive)` returns $\dot{\boldsymbol{x}}$ for the 17-entry state. It is a direct transcription of equations (7) to (9):

```
def body_rate_derivative(w, f, p):
    w_sigma = np.sum(p.prop_speeds(f))                    # sum of signed propeller speeds
    h = p.I_T @ w + p.Izz_P * w_sigma * E3                # angular momentum
    tau = p.body_torque(f * p.thrust_efficiency(w), w[2])
    return np.linalg.solve(p.I_B, tau - np.cross(w, h))   # eq. (rot)

def deriv(x, f_cmd, p, alive, motor_lag=True):
    q, w = x[IDX_Q], x[IDX_W]
    f_sat = saturate(f_cmd, p, alive)                     # 0 if stopped/failed, else clipped
    f = np.where(alive, x[IDX_F], 0.0)
    f_dot = np.where(alive, (f_sat - f) / p.sigma_mot, 0.0)   # motor lag
    R = quat_to_rot(q)
    acc = R @ E3 * np.sum(f) * p.thrust_efficiency(w) / p.m - G * E3
    q_dot = 0.5 * quat_mult(q, np.concatenate(([0.0], w)))
    w_dot = body_rate_derivative(w, f, p)
    return np.concatenate((x[IDX_V], acc, q_dot, w_dot, f_dot))
```

The boolean vector `alive` says which rotors work; a failed rotor's thrust is forced to zero. The same function `body_rate_derivative` is used by the simulator, by the equilibrium solver and by the linear models, so they cannot disagree with each other.

5.4 Integration and the sampled controller

`rk4_step` performs one classical Runge–Kutta step and then renormalizes the quaternion:

```
def rk4_step(x, f_cmd, dt, p, alive):
    k1 = deriv(x, f_cmd, p, alive)
    k2 = deriv(x + 0.5 * dt * k1, f_cmd, p, alive)
    k3 = deriv(x + 0.5 * dt * k2, f_cmd, p, alive)
    k4 = deriv(x + dt * k3, f_cmd, p, alive)
    x_new = x + dt / 6.0 * (k1 + 2 * k2 + 2 * k3 + k4)
    x_new[IDX_Q] /= np.linalg.norm(x_new[IDX_Q])
    return x_new
```

`simulate` runs the loop. The plant is integrated with step Δt (0.5 to 1 ms), but the controller is called only once per control period (1 to 5 ms) and its command is held constant in between. This is exactly a digital controller with a zero-order hold:

```
for k in range(n_steps + 1):
    t = k * dt
    if k % ctrl_every == 0:
        u = controller(t, x)          # sampled controller, held between calls
    ...                               # logging, optional early stop
    x = rk4_step(x, u, dt, p, alive_fn(t), motor_lag)
```

Every controller is a Python object called as `controller(t, x)` that returns four thrust commands. Controllers that keep internal state (filters, mode flags) provide a `reset` method. A supervisor may provide `alive(t)` to say when the rotor fails.

5.5 Equilibrium solver

`relaxed_hover(p, rho)` implements Section 4: a closed-form function for the rates and a one-dimensional root search for f_1:

```
def relaxed_hover(p, rho=0.5):
    def residual(f1):
        f = np.array([f1, rho * f1, f1, 0.0])
        w = _rates_for_thrust(f, p, w_guess)          # p = 0, r and q in closed form
        n = w / np.linalg.norm(w) * np.sign(w[2])
        return np.sum(f) * p.thrust_efficiency(w) * n[2] - p.weight
    f_lo = p.weight / (2 + rho)                       # thrust if the drone were level
    f1 = brentq(residual, f_lo * (1 + 1e-9), 6 * f_lo, xtol=1e-14)
    ...
```

When thrust loss is switched on, or the thrusts do not satisfy $f_1 = f_3$ and $f_4 = 0$, the closed form no longer applies, and `_rates_for_thrust` falls back to SciPy's `fsolve` on $\dot{\boldsymbol{\omega}} = 0$.

5.6 Linear models

The linear models are not typed in by hand. `failed_f` writes the 7-state equations, reusing `body_rate_derivative`, and `jacobians` differentiates them numerically by central differences with $\epsilon = 10^{-6}$:

```
def failed_f(s, u3, p):
    w, nx, ny = s[0:3], s[3], s[4]
    nz = np.sqrt(max(1.0 - nx * nx - ny * ny, 1e-12))
    n = np.array([nx, ny, nz])
    f = np.array([u3[0], u3[1], u3[2], 0.0])
    w_dot = body_rate_derivative(w, f, p)
    n_dot = -np.cross(w, n)
    vz_dot = np.sum(f) * nz / p.m - G
    return np.array([*w_dot, n_dot[0], n_dot[1], s[6], vz_dot])

def jacobians(fun, x0, u0, eps=1e-6):
    for i in range(n):          # A[:, i] = (f(x0 + e_i eps) - f(x0 - e_i eps)) / (2 eps)
        ...
```

This avoids transcription errors, and the tests check the result against finite differences of the same nonlinear function and against the paper's closed-form entries (for example $\bar{a}$). `add_motor_lag` then appends the three motor states.

5.7 Reproducibility

Every random number comes from `numpy.random.default_rng(seed)` with a fixed seed written in the experiment script. Every table and figure in this report is written by one script in `experiments/`. Success criteria are written in code (`metrics.py`) before the experiments are run, so they cannot be tuned afterwards. Appendix C lists the commands.

6 How we validated the simulator

Every later result depends on the simulator, so before using it we checked it in three independent ways:

1. **Physics tests**: compare the simulator with results derived by hand (exact formulas, conservation laws). These catch coding errors.
2. **Published numbers**: compare with the numbers printed in three papers. These catch modelling errors (a wrong sign, a missing term, a wrong parameter).
3. **Flight data**: compare with measurements from a real flight. This tells us how close the *model itself* is to reality.

All checks are automated tests that run with one command (`pytest`). At the time of writing, 46 tests pass and 1 is marked as an expected failure (explained in Section 6.2.1). A script in the `experiments` folder (`v0_validation_report.py`) also writes all comparisons into one table in `results/tables`.

6.1 Layer 1: physics tests against hand-derived results

Each test sets up a situation whose answer is known exactly, runs the simulator and compares. None of them compares the simulator with another run of itself.

Test	Exact reference result
Free fall	With no thrust: $z(t) = z_0 + v_{z0}t - \frac{1}{2}gt^2$, and horizontal motion at constant speed (agreement to 10^{-9}).
Thrust direction	Tilted drone, no torque: $\ddot{\boldsymbol{d}} = R\boldsymbol{e}_3 f_\Sigma/m - g\boldsymbol{e}_3$, with R built by hand from Euler angles.
Hover balance	With $f_i = mg/4$ all derivatives are zero.
Torque-free precession	No thrust, no drag: r stays constant and (p, q) rotate at the rate $\lambda = (I^T_{zz} - I^T_{xx})r/I^B_{xx}$ (Euler's equations solved in closed form; agreement to 10^{-8}).
Angular momentum and energy	For a torque-free rigid body the inertial angular momentum $RI\boldsymbol{\omega}$ and the kinetic energy $\frac{1}{2}\boldsymbol{\omega}^\top I\boldsymbol{\omega}$ must stay constant (relative error below 10^{-8}).
Reduced-attitude kinematics	For constant $\boldsymbol{\omega}$: $\boldsymbol{n}(t) = e^{-[\omega]\times t}\boldsymbol{n}(0)$, computed with the matrix exponential.
Yaw spin-up	Rotors 1 and 3 only, equal thrust: $p = q = 0$ stay zero and $I^B_{zz}\dot{r} = 2\kappa_\tau f - \gamma r$, so $r(t) = r_{ss}(1 - e^{-\gamma t/I^B_{zz}})$ with $r_{ss} = 2\kappa_\tau f/\gamma$.
Integration order	Halving the time step must reduce the error by about $2^4 = 16$ (RK4 is fourth order); the test accepts 12 to 20.
Motor lag	Step response $f(t) = u + (f_0 - u)e^{-t/\sigma}$.
Saturation rules	Command ≤ 0 gives 0; otherwise clipped to $[f_{\min}, f_{\max}]$; a failed rotor gives 0.
Paper's equations	An independent transcription of the paper's eqs. (12) to (14), written with signed propeller speeds instead of thrusts, equals the simulator at 50 random states.
IJRR model reduces to ours	Our separate implementation of the IJRR 2016 model (Section 6.2.2), with its extra effects switched off, equals the simulator at 50 random states.
Centre-of-mass motion	At the relaxed hover the vertical acceleration is zero and the horizontal acceleration is $g\sqrt{1 - n_z^2}/n_z$ (the paper's eqs. 18 to 20).

6.2 Layer 2: published numbers

6.2.1 Mueller and D'Andrea, ICRA 2014

This paper uses exactly our model and parameters, so we expect exact agreement up to the digits printed.

Reading. The two-rotor and three-rotor cases agree to every printed digit. The three-rotor case is a strong check because it exercises all gyroscopic and propeller terms with $p \neq 0$. In the one-rotor case the thrusts, the yaw rate and n_z agree within 0.5%, but q and n_y are 6% lower. We investigated this gap:

Table 4: Our equilibria against Mueller and D'Andrea, ICRA 2014.

Case	Quantity	Ours	Published	Difference
Two opposing rotors lost (eqs. 57–59)	$f_1 = f_3$ (N)	2.453	2.45	+0.10%
	propeller speed Ω_1 (rad/s)	−618.6	−619	+0.07%
	r (rad/s)	30.14	30.1	+0.14%
Three rotors lost (eqs. 62–64)	f_1 (N)	5.369	5.37	−0.02%
	propeller speed Ω_1 (rad/s)	−915.2	−915	−0.02%
	(p, q, r) (rad/s)	(14.68, 0, 32.99)	(14.7, 0, 33.0)	$\leq 0.11\%$
	$\boldsymbol{n}$	(0.407, 0, 0.914)	(0.41, 0, 0.91)	$\leq 0.8\%$
One rotor lost, $\rho = 0.5$ (eqs. 50–53)	$f_1 = f_3$, f_2 (N)	2.040, 1.020	2.05, 1.02	−0.5%, 0%
	r (rad/s)	18.81	18.89	−0.4%
	n_z	0.962	0.958	+0.4%
	q (rad/s)	5.37	5.69	−5.7%
	n_y	0.274	0.289	−5.1%
Fig. 3 (ρ sweep)	min $\|\boldsymbol{\omega}\|$ (rad/s) at ρ	18.62 at 0.675	19.0 at 0.655	−2.0%, +3.0%
	min f_1 (N) at ρ	2.029 at 0.579	2.04 at 0.563	−0.5%, +2.8%

- Inserting the paper's own published values ($\boldsymbol{f}$, $\boldsymbol{\omega}$) into the paper's own roll equation (its eq. 12) does *not* give zero: it leaves a roll torque of $-0.011\,\mathrm{N\,m}$, about 7% of the torque from rotor 2. So the published one-rotor values are not an exact equilibrium of the paper's own model with its stated parameters.
- The published inertias have only two significant figures. Changing each by $\pm 0.05 \times 10^{-3}$ moves our q between 5.07 and 5.69 rad/s, so the published value lies at the edge of what rounding allows; but the same parameters reproduce the other two cases exactly.

We therefore treat this as an inconsistency in that one published case, not as an error in our model. The test for the published q is kept and marked as an expected failure, so the discrepancy stays visible.

Analytic checks. The paper's closed form for the yaw rate (its eq. 21) equals our solver. Its coupling constant $\bar{a}$ (eq. 28) equals the (p, q) entry of our numerical Jacobian. Its controllability condition (eqs. 37 to 39) says that with two opposing rotors lost, the roll rate becomes uncontrollable exactly when $\bar{a} = 0$; our controllability rank drops from 7 to 6 exactly there.

The sweep over the thrust ratio ρ (Figure 2) reproduces the shape of the paper's Fig. 3; the minima agree within 2% in value and 3% in location. Both minima are shallow, so their location is sensitive to small differences.

6.2.2 Mueller and D'Andrea, IJRR 2016

This paper uses the same drone with a more detailed model (air-relative thrust, quadratic drag on all axes, propeller speeds as inputs, the propeller acceleration term kept). To check it we wrote a *separate* implementation, `validation.py`, deliberately in different variables (propeller speeds instead of thrusts), so that it does not share code with the simulator. For each of the five published cases we give it the published propeller speeds and solve for the body rates at which the rotation stops changing.

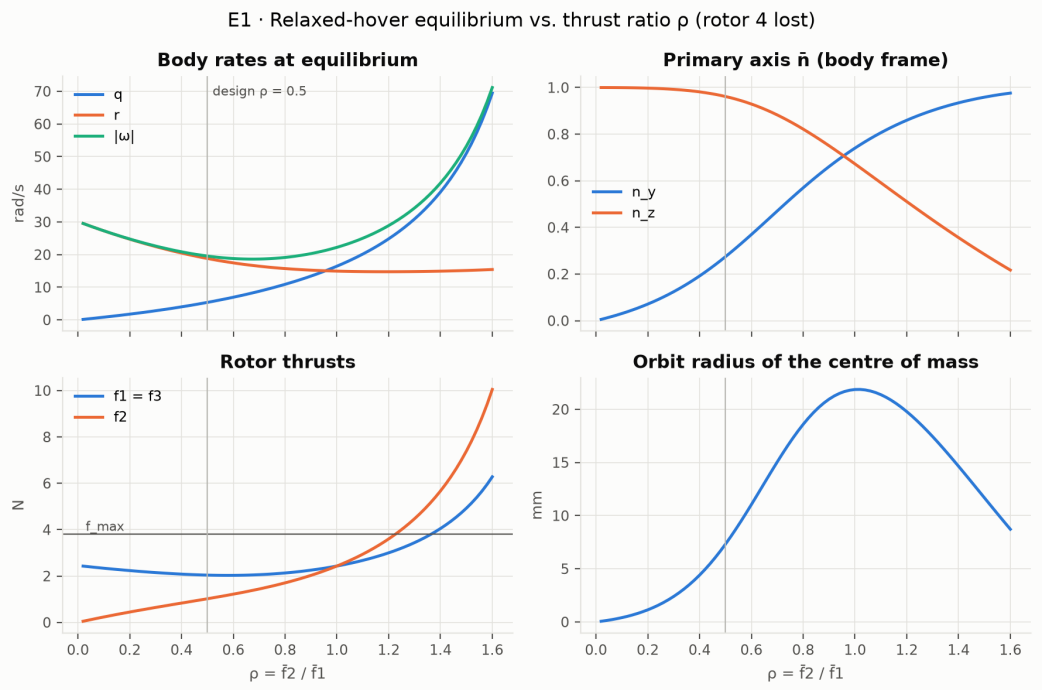


Figure 2: The one-rotor-loss equilibrium as a function of the thrust ratio $\rho = f_2/f_1$ (reproduces the paper's Fig. 3).

Table 5: Hover solutions of IJRR 2016 (Sec. 5.2) against our independent implementation. The last column is $f_\Sigma n_z/(mg)$, which must be 1 at a hover solution.

Case	Published Ω (rad/s)	Published $\boldsymbol{\omega}$	Ours	Force ratio
No failure	(438, 438, 438, 438)	(0, 0, 0)	(0, 0, 0)	1.003
Rotor 4 lost	(585, 362, 585, 0)	(0.20, 4.30, 19.50)	(0.15, 4.19, 19.47)	1.002
Rotors 2, 4 lost	(643, 0, 643, 0)	(0, 0, 24.50)	(0, 0, 24.50)	1.001
Rotors 3, 4 lost	(1067, 218, 0, 0)	(26.00, 0, 23.30)	(24.94, 0.06, 23.57)	0.995
Rotors 2, 3, 4 lost	(1103, 0, 0, 0)	(28.00, −1.60, 24.50)	(26.73, −1.45, 24.93)	0.994

Reading. With the published parameters, the body-rate vectors agree within 4% and the force balance within 0.6%. The two cases with very high rotation rates are the most sensitive to the drag and inertia values, which are printed with one or two significant figures. As a sensitivity check only, $I_{xx} = 2.75 \times 10^{-3}$ (the upper end of the rounding interval of the printed 2.7×10^{-3}) brings all five cases within 0.7%; because this value was chosen by looking at the same cases, it is not an independent validation.

One number is especially informative. With two opposing rotors lost, the paper's propeller speed is 643 rad/s at a body spin of 24.5 rad/s. If thrust depended on the speed relative to the body, $\kappa_f \cdot 643^2$ would be 8% more than $mg/2$; with the speed relative to the air, $\kappa_f(643-24.5)^2 = mg/2$ exactly. So air-relative thrust is real and matters. Our simulator (the ICRA model) does not include it: at the one-rotor equilibrium it would change rotors 1 and 3 by about −7% and rotor 2 by about +10%. This is a known limitation (Section 12).

6.2.3 Sun et al., T-RO 2021: a second airframe

To test the equilibrium solver on a different drone, we used the Parrot Bebop 2 parameters of Sun et al. [16] (their Table I in the arXiv version, called Table III in their text: $m = 0.410$ kg, $l = 0.145$ m, and so on). With two opposing rotors lost, their eq. (60) gives $\bar{r} = mg\kappa_\tau/\gamma$ and propeller speed $\sqrt{mg/(2\kappa_f)}$.

Quantity	Ours	Eq. (60) with the parameter table	Printed in the text
$\bar{r}$ (rad/s)	26.81	26.81	26.4
propeller speed (rad/s)	1029	1029	1015

Our solver equals the paper's own formula exactly. The numbers printed in the paper's text are about 1.5% below its own formula; this difference is internal to the paper.

6.3 Layer 3: flight data

Spin-up time. In the ICRA flight (their Fig. 4), propeller 4 is switched off and the drone flies on propellers 1 and 3 only, without attitude feedback, until the rotation rate reaches 10 rad/s, which took 0.62 s. During this phase the roll and pitch torques are zero (rotors 1 and 3 are opposite each other with equal thrust), so only the yaw equation acts:

$$I_{zz}^B \dot{r} = 2\kappa_\tau f - \gamma r \quad \Rightarrow \quad r(t) = r_{ss}(1 - e^{-\gamma t / I_{zz}^B}), \quad r_{ss} = \frac{2\kappa_\tau f}{\gamma}, \tag{20}$$

so the time to reach 10 rad/s is $t_{10} = -\frac{I_{zz}^B}{\gamma} \ln(1 - \frac{10}{r_{ss}})$. The answer depends strongly on the thrust f of the two rotors. The paper's text says only that the vehicle "uses only the two opposing propellers" during this phase; the thrust is not printed, so we read it from its Fig. 4: rotor 2 is at zero and rotors 1 and 3 at a constant 2.95 N or so (more than $mg/2 = 2.45$ N).

Spin-up thrust	Simulator	Closed form	Flight
2.95 N (read from Fig. 4)	0.65 s	0.64 s	0.62 s
2.45 N ($mg/2$)	0.81 s	0.80 s	not flown

At the flight thrust the model is within about 4% of the flight. Because the thrust is read from a plot, the detection-delay study (Section 10) is reported for both thrusts.

Steady spin rate and height. The published models over-predict the spin of the real drone. In the ICRA one-rotor flight the measured body rates in hover were $(2.1, 3.7, 13.7)$ rad/s (their eq. 55), about 14.3 rad/s against about 19.7 rad/s predicted by the same model that our simulator implements (ours: 19.6 rad/s at $\rho = 0.5$), about 27% less. In the IJRR flight the spin was 15.8 rad/s against 19.9 rad/s in simulation (their Sec. 5.4), about 20% less; those authors suggest extra torques from the motion of the propeller centres as a possible cause. In the ICRA two-rotor flight the drone stayed about 2.4 m below its set-point, which the authors attribute to the propellers producing less thrust at high body rates. None of these effects is in the nominal model. They are why our truth plants contain a thrust loss at high rotation rate: the learned residual of Section 8 is meant to capture effects of this kind.

6.4 Summary of the validation

In plain words

The simulator agrees with the papers wherever the papers are internally consistent, and the only disagreements are explained: one published case that does not satisfy the paper's own equation, two cases whose parameters are printed with very few digits, and two real-world effects (air-relative thrust and thrust loss while spinning) that the paper's own model also leaves out. We can therefore trust the simulator as a faithful implementation of the published model, and we know in which ways the published model differs from the real drone.

7 Linear analysis and LQR controllers

7.1 Controllability: can three rotors steer the failed drone?

Method. For both linear models we computed three things (experiment E2, `analysis.py`):

1. The rank of the controllability matrix $\mathcal{C} = [B, AB, \ldots, A^{n-1}B]$. Its columns differ in size by many orders of magnitude (each multiplication by A scales them by about 20), which can make a numerical rank test unreliable. We therefore divide every column by its length before computing the singular values ("scaled rank").
2. The PBH test for every eigenvalue λ of A: the smallest singular value of $[\lambda I - A,\ \tilde{B}]$, where $\tilde{B}$ is B rescaled to the size of A. A value near zero would mark an uncontrollable mode.
3. The controllability Gramian over 1 s, $W_c(T) = \int_0^T e^{As} B B^\top e^{A^\top s} ds$. It is computed exactly with Van Loan's method: the matrix exponential of the block matrix $\begin{pmatrix} -A & BB^\top \\ 0 & A^\top \end{pmatrix} T$ contains W_c in its blocks. This works even for the undamped modes, where the infinite-time Gramian does not exist.

Result. Both models are controllable. The failed model has rank 7 of 7, and every mode passes the PBH test (smallest values between 0.016 and 0.28, none near zero). The Gramian's smallest eigenvalue is 0.54 and its condition number 7.6×10^3: some directions need much more effort than others, but none is out of reach. This reproduces the paper's controllability result; it is not a new finding.

7.2 Observability: what can the sensors see?

Method. A controller needs the state, so we asked which sensors make it observable. For each sensor set we built the output matrix C, checked the rank of the observability matrix, and computed the 1 s observability Gramian. Different sensors measure different units with different noise, so before computing the Gramian we divided each output by an assumed noise level (one standard deviation): gyroscope 0.005 rad/s, position 0.01 m, attitude 0.01 rad, magnetometer heading 0.02 rad, tilt estimate 0.01, barometer 0.1 m. The Gramian is then dimensionless, and its smallest eigenvalue says how well the weakest direction is seen *relative to the noise*.

Table 6: Observability of the failed drone's linear model (7 states) for four sensor sets.

Sensors	Rank	Cannot see	Smallest eigenvalue of $W_o(1\,\mathrm{s})$
Gyroscope (p, q, r)	3/7	n_x, n_y, z, v_z	0
Gyroscope + tilt estimate (n_x, n_y)	5/7	z, v_z	0
Gyroscope + barometer (z)	7/7	nothing	2.4×10^{-3}
Gyroscope + tilt + barometer	7/7	nothing	6.6

Reading. Gyroscopes measure rotation rates only. Because the rate equations do not depend on $\boldsymbol{n}$ (the first three rows of A have zeros in the $\boldsymbol{n}$ columns), the rates carry no information about the tilt, so gyroscopes alone see neither tilt nor height. A small finding of ours: because the relaxed hover is *tilted*, the vertical acceleration depends on the tilt to first order ($A_{v_z n_y} = -2.91$), so a barometer that measures height also reveals the tilt, and gyroscope plus barometer is enough in principle. With realistic noise, however, the smallest eigenvalue of the noise-normalized Gramian is 2.4×10^{-3}, against 6.6 with a direct tilt measurement: about 2,700 times smaller, which corresponds to roughly 50 times larger estimation error (standard deviation) in the weakest direction. It is a structural remark, not a practical estimator; real estimators for the spinning drone use inertial sensors together with vision [42] or a magnetometer [43]. For the healthy

drone, gyroscope plus position leaves the heading unobservable (heading enters the motion only at second order near hover), as expected, and a magnetometer fixes it.

7.3 LQR controllers

Healthy LQR. On the 12-state hover model we used Bryson's rule with largest acceptable errors 0.5 m horizontal and 0.3 m vertical position, 1 m/s horizontal and 0.5 m/s vertical speed, 0.2 rad roll and pitch, 0.5 rad yaw, 2 rad/s body rates, and 0.5 N per rotor. A factor α multiplies all state weights, to make the controller more ($\alpha > 1$) or less ($\alpha < 1$) aggressive. The controller converts the quaternion to Euler angles, wraps the yaw error to $[-\pi, \pi]$, and clips the commands to $[f_{\min}, f_{\max}]$.

Failure LQR. On the 10-state model (7 states and 3 motor states) we used the paper's weights: 1 on p and q, 20 on n_x and n_y, 0 on the motor states, and $1\,\mathrm{N}^{-2}$ on each input. The paper's reduced state does not contain r; we include it with weight 0 (our choice), and we added 4 on height and 1 on vertical speed, because we also control altitude. The control law is

$$\boldsymbol{u} = \bar{\boldsymbol{f}} - K\,(\boldsymbol{s} - \bar{\boldsymbol{s}},\ \hat{\boldsymbol{f}} - \bar{\boldsymbol{f}}), \qquad \text{clipped to } [f_{\min}, f_{\max}], \tag{21}$$

where $\hat{\boldsymbol{f}}$ is an estimate of the motor thrusts. Motor thrusts are not measured, so the controller reconstructs them by passing its own commands through the same first-order lag: $\hat{\boldsymbol{f}}_{k+1} = \hat{\boldsymbol{f}}_k + (1 - e^{-\Delta t/\sigma})(\boldsymbol{u}_k - \hat{\boldsymbol{f}}_k)$. When the controller is switched on after a failure, this estimate is initialized from what the flight computer knows (a filter of the commands sent before), not from the true thrust, which it cannot measure.

In the code: the failure LQR

```
def __call__(self, t, x):
    s = self.state7(x)                          # (p, q, r, n_x, n_y, z, v_z) from the 17-state
    e = s - self.base.x_trim                    # deviation from the relaxed hover
    e[5] -= self.z_ref
    e = np.concatenate((e, self.f_est - self.eq.f[:3]))     # add motor-state estimate
    u3 = np.clip(self.eq.f[:3] - self.K @ e, self.p.f_min, self.p.f_max)
    self.f_est += (1 - np.exp(-dt / self.p.sigma_mot)) * (u3 - self.f_est)   # exact lag
    step
    return np.array([u3[0], u3[1], u3[2], 0.0])          # rotor 4 gets nothing
```

Delay margin. To find how much pure input delay h each loop tolerates, we discretize the linear loop exactly at 1 ms and append a chain of $h/1$ ms memory states that pass the command along (a "delay line"). The loop is stable if all eigenvalues of this larger matrix lie inside the unit circle. We increase h in 1 ms steps from zero and stop at the first unstable value.

Table 7: LQR designs (experiment E3).

Controller	Slowest closed-loop poles	Delay margin
Healthy LQR (12 states)	$-1.72 \pm 1.02j$, -1.89, $-1.94 \pm 1.56j$	40 ms
Failure LQR (7 + 3 motor states)	-1.09, $-2.01 \pm 1.04j$, $-3.80 \pm 19.68j$	18 ms
Failure LQR without motor states	-1.09, $-2.01 \pm 1.04j$, $-3.83 \pm 19.66j$	18 ms

Results. All closed-loop poles have negative real parts. The least-damped mode of the failure LQR is the oscillation near the spin frequency, $-3.80 \pm 19.68j$, with damping ratio about 0.19. The failure LQR tolerates only 18 ms of input delay, against 40 ms for the healthy one: the spinning drone is a much less forgiving system. The design without motor states remains stable

when the real 15 ms motor lag is present (largest real part -1.10), but we follow the paper and include the motors.

On the full nonlinear model with motor lag and thrust limits, we started the failure LQR at the relaxed hover with rate errors $(1, -1, -3)$ rad/s, tilt errors $(0.08, -0.05)$ and 1 m below the set-point (Figure 3). It returned to the equilibrium: the tilt settled within 0.01 in 2.1 s, the height within 5 cm in 1.8 s and the yaw rate within 0.5 rad/s in 2.5 s. After 6 s the remaining errors were 1.6×10^{-4} in tilt and 0.012 rad/s in rate. The healthy LQR, started 0.5 m off and tilted by up to 0.4 rad, returned to within 0.25 mm in 5 s (Figure 4).

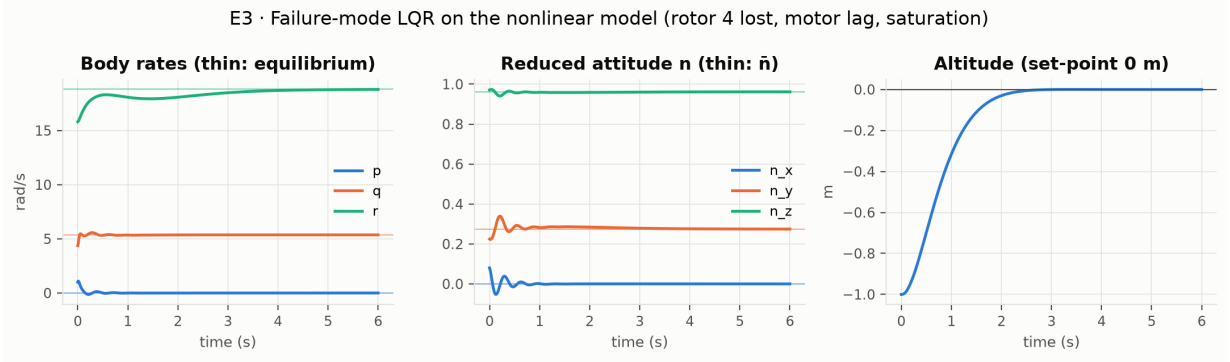


Figure 3: Failure LQR on the nonlinear model (rotor 4 lost, motor lag, thrust limits), starting from a perturbed relaxed hover 1 m below the set-point. Thin lines: equilibrium values.

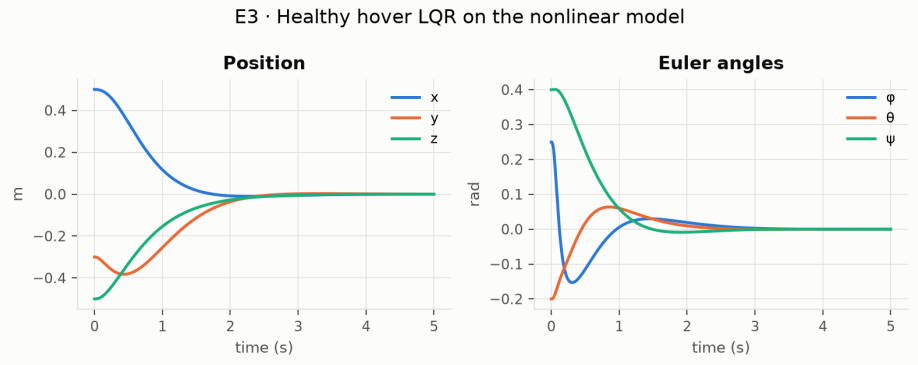


Figure 4: Healthy LQR on the nonlinear model from an offset, tilted start.

8 Learning the model error: residual Koopman models (RQ1)

This section answers the prediction half of RQ1: which model best predicts how the failed drone will move? We compare three predictors trained and tested on exactly the same data (experiment E6).

8.1 Hypotheses

Before running the experiment we wrote down what we expected (from the proposal):

H1. A Koopman model of the full dynamics, learned from failed-drone data, has eigenvalues outside the unit circle and loses to a trivial "nothing changes" predictor at long horizons. (Our proposal found this for a healthy drone.)

H2. The residual model has a lower multi-step prediction error than both the physics model and the full Koopman model.

H4. The residual model reaches a given error with less data than the full Koopman model. (H3 concerns MPC and is tested in Section 9.)

8.2 Data

Why a separate truth plant. The data come from truth plant A (Section 4.10), not from the nominal model. Otherwise the physics model would already be perfect and there would be nothing to learn.

How episodes are generated. Each episode lasts 4 s with rotor 4 failed:

1. The drone starts near the relaxed hover, with random deviations drawn uniformly: body rates ± 2 rad/s, tilt ± 0.15 in n_x and n_y, height and vertical speed ± 0.5.
2. It is flown by the failure LQR designed on the *nominal* parameters. A learned model needs to see the drone respond to many different inputs, so we add a random *excitation*: every 50 ms a new random offset, uniform in ± 0.35 N, is added to each of the three thrust commands (then clipped to the limits).
3. The simulator runs at 1 ms; the controller and the recorded data run at $T_\mathrm{s} = 5$ ms.
4. At every sample we record the measured state as a deviation from the relaxed hover, $\boldsymbol{x} = (p, q, r, n_x, n_y, z, v_z) - \bar{\boldsymbol{s}}$; the command deviation $\boldsymbol{u} = (f_1, f_2, f_3) - \bar{\boldsymbol{f}}$; and $\hat{\boldsymbol{f}}$, the nominal first-order filter of the commands. The true motor thrust is not used, because a real flight computer does not measure it.
5. Episodes in which the tilt deviation exceeds 0.6 (the drone has left the region of interest) are discarded.

Training uses 40 episodes (32,000 samples, seed 11). Testing uses 10 new episodes from plant A (seed 12) and 10 from the held-out plant B (seed 13). A second training set of 40 episodes, each from a randomly drawn plant (seed 21), tests whether varied training data helps.

8.3 The three predictors

1. Analytical (physics only). The 10-state linear model of Section 4.9 (7 states and 3 motor states, nominal parameters), discretized exactly with a zero-order hold at 5 ms:

$$\boldsymbol{s}_{k+1} = A_d \boldsymbol{s}_k + B_d \boldsymbol{u}_k, \qquad \boldsymbol{s} = (\boldsymbol{x}, \hat{\boldsymbol{f}}) \in \mathbb{R}^{10}. \tag{22}$$

It uses no data at all.

2. Full-state EDMDc. Everything is learned from data [13, 14]. We take the 9-number vector $\boldsymbol{\xi} = (p, q, r, n_x, n_y, v_z, \hat{f}_1, \hat{f}_2, \hat{f}_3)$; height z is left out because it is just the integral of v_z. Each entry is *standardized* (the training mean subtracted and the result divided by the training standard deviation), giving $\tilde{\boldsymbol{\xi}}$. The dictionary is

$$\psi(\boldsymbol{\xi}) = (\ 1,\ \ \tilde{\xi}_1, \ldots, \tilde{\xi}_9,\ \ \tilde{\xi}_1^2, \tilde{\xi}_1\tilde{\xi}_2, \ldots, \tilde{\xi}_9^2\ ), \tag{23}$$

that is, a constant, the 9 standardized values and all 45 products of two of them: $1 + 9 + 45 = 55$ functions. We then fit the lifted linear model

$$\psi(\boldsymbol{\xi}_{k+1}) \approx A\,\psi(\boldsymbol{\xi}_k) + B\,\boldsymbol{u}_k \tag{24}$$

by ridge regression over all training pairs (regularization $\lambda = 10^{-6}$). To predict, we lift the starting state once, iterate $\boldsymbol{\zeta} \leftarrow A\boldsymbol{\zeta} + B\boldsymbol{u}$ (we write $\boldsymbol{\zeta}$ for the lifted state here, to avoid confusion with the height z), read $\boldsymbol{\xi}$ back from the 9 linear coordinates of $\boldsymbol{\zeta}$, and integrate v_z with the trapezoid rule to get the height z.

3. Residual EDMDc (proposed). Keep the physics model and learn only its one-step error. For every training pair we compute what the physics model would have predicted and the difference to what really happened:

$$\boldsymbol{e}_{k+1} = \boldsymbol{x}_{k+1} - \left[A_d \boldsymbol{s}_k + B_d \boldsymbol{u}_k\right]_{1:7}. \tag{25}$$

We regress $\boldsymbol{e}_{k+1}$ on the same kind of quadratic dictionary, now built from $(\boldsymbol{\xi}_k, \boldsymbol{u}_k)$ (the same 9 values plus the 3 commands, standardized; $1 + 12 + 78 = 91$ functions), with ridge regularization $\lambda = 10^{-3}$, giving a matrix W. The prediction is

$$\boldsymbol{x}_{k+1} = \left[A_d \boldsymbol{s}_k + B_d \boldsymbol{u}_k\right]_{1:7} + \mathrm{clip}\left(W^\top \psi(\boldsymbol{\xi}_k, \boldsymbol{u}_k),\ \pm \boldsymbol{d}_{\max}\right), \tag{26}$$

where $\boldsymbol{d}_{\max}$ is the 99.5th percentile of the absolute training residuals, per state. The clipping, borrowed from Todde et al. [3], prevents the learned part from producing a correction larger than anything seen in training, for example when the drone is far from the training data.

Why we call it a residual Koopman model. Strictly, this model uses the EDMD dictionary and least squares to learn a *one-step* correction; unlike the full-state EDMDc model it does not propagate a lifted linear state, and the input enters it nonlinearly (through products such as $\xi_i u_j$). It is closer to a polynomial regression of the error than to a linear Koopman predictor, and we keep the name only because it follows the same construction as the residual models in the literature [3, 5]. What keeps the MPC problem convex is not linearity of this model but the way it is used: it is evaluated outside the optimization (Section 9).

In plain words

The physics model provides most of the prediction: it already knows that the drone spins, how fast, and how it responds to each rotor. The learned part only has to describe a small correction, such as "the drone sinks a little faster than the physics says when it spins fast", and that correction is much easier to learn than the whole motion.

In the code: fitting the residual

```
for X, fe, U in zip(data.x, data.fe, data.u):          # every training episode
    for k in range(len(U)):
        x1n, _ = nom.step(X[k], fe[k], U[k])            # physics prediction
        E.append(X[k + 1] - x1n)                        # what it got wrong
        F.append(np.concatenate((X[k, XI_IDX], fe[k], U[k])))
dic = Dictionary(F.mean(0), F.std(0) + 1e-9)           # standardize, quadratic features
W = ridge(dic(F), E, lam)                               # (Phi'Phi + lam I)^-1 Phi'E
d_max = np.percentile(np.abs(E), 99.5, axis=0)          # clipping level
```

4. No-change baseline. The prediction is simply the starting state, held forever. Any model worse than this is useless at that horizon.

8.4 How prediction error is measured

From each test episode (800 samples) we start a prediction every 40 samples (0.2 s), as long as the longest horizon (600 samples) still fits: 5 starts per episode, all in the first 0.8 s, so 50 starts per test set. Each predictor is given the true starting state and the recorded inputs, and it predicts *open loop*, that is, without seeing any later measurement. We compare the prediction with the true trajectory at horizons from 5 ms to 3 s. Each state's error is divided by that state's standard deviation in the training data (so rad/s and metres can be combined), squared, averaged over all start times and all 7 states, and square-rooted:

$$\mathrm{NRMSE}(h) = \sqrt{\frac{1}{7 N_{\text{starts}}} \sum_{\text{starts}} \sum_{i=1}^{7} \left(\frac{\hat{x}_i(h) - x_i(h)}{\mathrm{std}_i}\right)^2}. \tag{27}$$

(To keep a diverging model from producing overflow, each squared error is capped at 10^{12} before averaging.) A value of 1 means the error is as large as the typical variation of the states; the no-change baseline sits at about 2.4.

8.5 Results

Table 8: Normalized multi-step prediction error (lower is better), models trained on family A.

	Test family A				Test family B (held out)		
Predictor	20 ms	0.1 s	1 s	2 s	0.1 s	1 s	2 s
Analytical	0.65	3.17	7.09	7.93	4.31	7.94	6.95
Full-state EDMDc	**0.04**	0.17	0.91	3.45	6.09	26.2	131
Residual EDMDc	0.10	**0.15**	**0.64**	2.71	5.98	19.9	24.0
No change	0.81	2.42	2.38	2.51	2.51	2.44	2.41



Figure 5: Multi-step prediction error against horizon (both axes logarithmic). Top: trained on family A. Bottom: trained on randomized plants. Left: tested on family A; right: tested on the held-out family B.

H2 (residual is best): supported on the training plant. On new episodes from plant A, the residual model is the most accurate of the three models at every horizon from 100 ms on, and the most accurate predictor overall from 100 ms to 1 s (Table 8, Figure 5 top left). At 1 s its error is 0.64, against 0.91 for full-state EDMDc and 7.09 for the physics model. At short horizons (5 to 50 ms) the full-state model is better, by a factor of 2.5 at 20 ms (0.04 against 0.10). The physics model is worse than the no-change baseline from about 100 ms on: its undamped modes and the unmodelled thrust loss make it drift away. At 2 s even the residual model is behind the no-change baseline, so long open-loop horizons need feedback, which is what MPC provides by re-planning every 5 ms.

H1 (full model is unstable): supported. The learned 55×55 matrix A of the full-state model has 4 eigenvalues outside the unit circle (spectral radius 1.0070, Figure 6). The true open-loop model has undamped modes that lie exactly on the unit circle in discrete time, so this is a small outward push of modes that are only marginally stable in reality, not a qualitatively different defect. Over many steps, however, it makes errors grow, and the model falls behind the no-change baseline by 2 s and explodes at 3 s (error 14.1). This repeats, in the failed regime, what our proposal found for a healthy drone (with full-state EDMDc, 10 of 22 eigenvalues were outside the unit circle).

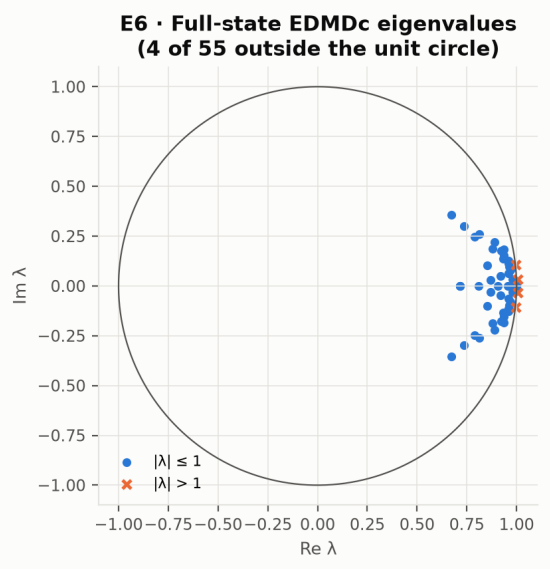


Figure 6: Eigenvalues of the learned full-state EDMDc matrix. Crosses lie outside the unit circle, so their modes grow.

H4 (less data): supported, with one exception. Figure 7 repeats the training with 2, 5, 10, 20 and 40 episodes and measures the error at a 0.5 s horizon. The residual model trained on only 10 episodes (error 0.42) is already better than the full-state model trained on all 40 (0.48). This matters because data from failed flights is scarce: such flights often end in a crash. The same advantage of residual models has been reported for soft robots, ground vehicles and quadrupeds [4, 5, 38]; we confirm it in the spinning regime. The curves are not monotone: at 5 episodes the full-state model is slightly better (0.93 against 1.02).

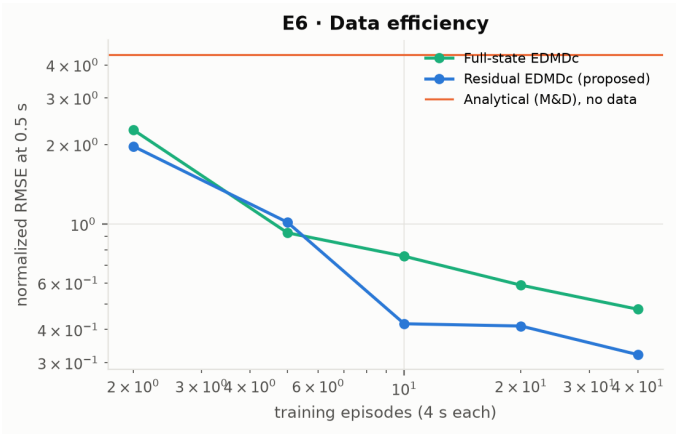


Figure 7: Prediction error at 0.5 s against the number of training episodes (family A). The flat line is the physics model, which uses no data.

Generalization: not achieved. On the held-out plant B, both models trained on plant A are *worse* than the physics model (Table 8, right). The learned correction is specific to the plant it saw: it "corrects" B as if it were A, which makes things worse. Training on randomized plants, one random plant per episode, does not solve this. It helps the residual model on B only up to about 100 ms (error 3.68 against 4.31 for the physics model) and makes both learned models worse on A (the full-state model trained this way even diverges, spectral radius 1.028). The likely reason is that a single state sample does not reveal which plant the drone is: the motor time constant and the thrust loss show only in how the state changes over several steps. A static regression then learns an average correction that fits no plant well. This points to online adaptation: because the residual is linear in W, it can be updated during flight by recursive least squares. Online-adapted Koopman models exist for healthy quadrotors [39] and for loss-of-effectiveness faults [36].

8.6 What this means for RQ1

In plain words

On the drone it was trained on, learning only the error of the physics model gives the best predictions from 0.1 to 1 s, and with 10 training episodes it is more accurate than learning everything from 40 episodes. But the learned correction does not carry over to a drone with different parameters. In its current form it is a calibration for one vehicle, not a general model of rotor failure.

9 Model predictive control with the learned residual (RQ1)

This section answers the control half of RQ1 and tests hypothesis **H3**: a residual Koopman MPC removes the steady error that model mismatch causes in a controller designed on the physics model (experiment E7).

9.1 The MPC problem

At every sample (5 ms) the controller measures the current state, builds $\boldsymbol{s}_0 = (\boldsymbol{x}, \hat{\boldsymbol{f}})$ and solves

$$\begin{aligned} \min_{\boldsymbol{u}_0,\dots,\boldsymbol{u}_{N-1}} \quad & \textstyle\sum_{j=0}^{N-1} \left(\boldsymbol{s}_j^\top Q\, \boldsymbol{s}_j + \boldsymbol{u}_j^\top R\, \boldsymbol{u}_j\right) + \boldsymbol{s}_N^\top P_N\, \boldsymbol{s}_N \\ \text{subject to} \quad & \boldsymbol{s}_{j+1} = A_d \boldsymbol{s}_j + B_d \boldsymbol{u}_j + \hat{\boldsymbol{d}}, \qquad j = 0, \dots, N-1, \\ & f_{\min} - \bar{f}_i \;\le\; u_{j,i} \;\le\; f_{\max} - \bar{f}_i. \end{aligned} \tag{28}$$

- The prediction model is the 10-state analytical model of Section 8.
- The horizon is $N = 20$ steps of 5 ms, so the controller looks 100 ms ahead.
- Q and R are the same weights as the failure LQR, so the two controllers aim for the same thing.
- The *terminal cost* P_N is the solution of the discrete-time Riccati equation for (A_d, B_d, Q, R). It stands in for the cost after the horizon; with it, the MPC behaves like the discrete LQR whenever the limits are not active, which our tests check.
- The input limits are the motor thrust limits, written as deviations from the trim.

Only the first input $\boldsymbol{u}_0$ is applied; at the next sample the problem is solved again from the new measurement.

9.2 Where the learned residual enters

The **analytical MPC** uses $\hat{\boldsymbol{d}} = 0$. The **residual Koopman MPC** evaluates the learned residual once per sample, at the current state and previous input, *outside* the optimization, and adds it to every predicted step as a known constant $\hat{\boldsymbol{d}}$ (a "measured disturbance"), as Todde et al. do [3]. The learned part is therefore not a decision variable, the constraints stay linear, and (28) stays a convex QP that solves reliably every time. The price is that the optimizer does not know how the residual would change with its own choice of inputs or along the predicted trajectory: the correction is frozen at its current value for the whole 100 ms horizon.

In plain words

The physics model plans the next 100 ms; the learned residual tells it "at the moment the real drone drifts this much more per step than you think", and the plan compensates for that drift.

In the code: building and solving the QP once per sample

```
# built once: CVXPY parameters x0 and d, variables X (states) and U (inputs)
for j in range(N):
    cons += [X[:, j + 1] == Ad @ X[:, j] + Bd @ U[:, j] + self.d,
             U[:, j] >= lo, U[:, j] <= hi]
    cost += cp.quad_form(X[:, j], Q) + cp.quad_form(U[:, j], R)
cost += cp.quad_form(X[:, N], PN)                        # DARE terminal cost

# every sample
d[:7] = self.residual.residual(x7, self.fe, self.u_prev) # learned part, outside the QP
self.x0.value, self.d.value = np.concatenate((x7, self.fe)), d
self.prob.solve(solver=cp.OSQP, warm_start=True, eps_abs=1e-5, eps_rel=1e-5)
u = self.U.value[:, 0]                                    # apply the first move only
```

The problem is modelled with CVXPY [46] and solved with OSQP [47], an operator-splitting QP solver; "warm start" reuses the previous solution as the starting guess.

9.3 Experiment: steady altitude error under mismatch

All three controllers (LQR, analytical MPC, residual Koopman MPC) are designed on the nominal parameters and flown on the mismatched truth plants. The residual is the one trained on family A in Section 8. Each run starts 1 m below the set-point near the relaxed hover and lasts 8 s; the steady error is the mean height over the last 2 s. We use one run per plant and controller, because the steady state does not depend on the starting point. To see which effect causes the error, we also test each family without its thrust loss, and the nominal plant with only that family's thrust loss.

Table 9: Steady altitude error near the relaxed hover (set-point 0 m). The last column compares the *magnitude* of the residual-MPC error with the LQR error; for plant B (full) the sign also changes.

Plant	LQR	MPC, analytical	MPC, residual	Change
A (full)	−0.285 m	−0.286 m	**−0.193** m	−32%
A without thrust loss	−0.154 m	−0.154 m	−0.066 m	−57%
Nominal + A's thrust loss only	−0.100 m	−0.101 m	−0.033 m	−67%
B (full, held out)	−0.029 m	−0.033 m	+0.022 m	−25%
B without thrust loss	+0.112 m	+0.111 m	+0.164 m	+47%
Nominal + B's thrust loss only	−0.174 m	−0.177 m	−0.107 m	−39%

Results. On plant A, LQR and the analytical MPC both settle 0.29 m too low; they are practically identical, because near the equilibrium the thrust limits are not reached and the MPC

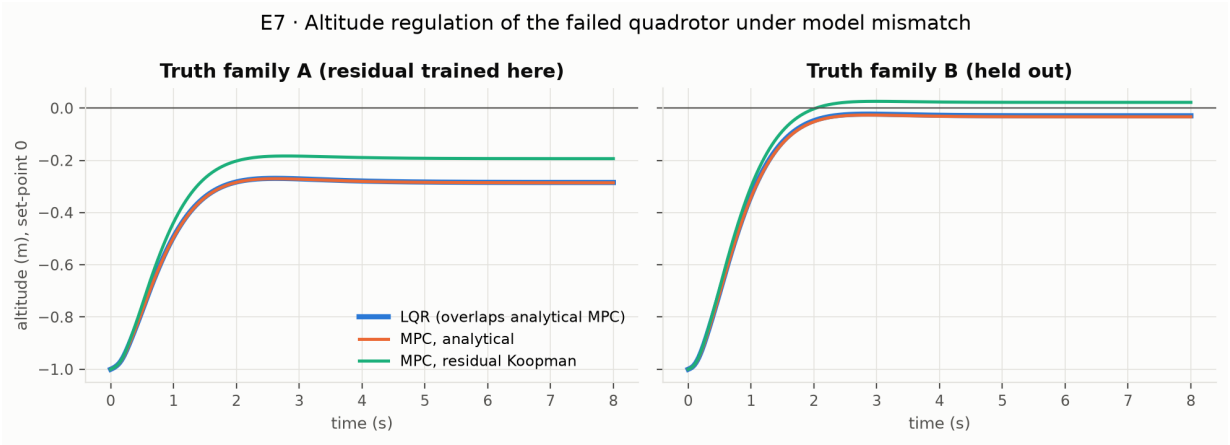


Figure 8: Height of the failed drone under model mismatch for the three controllers (LQR and analytical MPC overlap). Left: family A, on which the residual was trained. Right: held-out family B.

then acts like the LQR. The thrust loss alone causes 0.10 m and the inertia and motor mismatch alone 0.15 m; the effects are not exactly additive (together 0.29 m). The residual Koopman MPC settles 0.19 m low, a 32% smaller error, and it reduces the error for each separate effect of plant A. On the held-out plant B the effects partly cancel. The residual MPC still reduces the error for the full plant and for its thrust loss alone, but when B's thrust loss is removed it makes the error 47% *larger*: the correction learned on A pushes in the wrong direction for that plant. This is the generalization problem of Section 8 again. H3 is therefore only partly supported: on the training plant the error is reduced by 32%, not removed, and on a held-out variant it grows. The errors on plant B are only a few centimetres, so their percentages should not be over-read.

The residual reduces the error but does not remove it. The classical way to remove a steady error caused by a constant mismatch is integral action, or an "offset-free" MPC that estimates a constant disturbance from the measurements; in theory it removes such an error completely. This is the most important missing baseline: until it is compared, the 32% reduction shows that the residual carries useful information, not that it is the best way to remove the bias. A residual could still help where integral action is slow, for example during transients.

Computation time. The QP solver needs a median of 0.73 ms per sample. Through the CVXPY modelling layer the median wall-clock time is 4.4 ms, which leaves almost no margin to the 5 ms sample period. A real-time version would call the solver directly.

10 How long may fault detection take? (RQ2, empirical)

10.1 The failure timeline

Figure 9 shows what happens around a failure. At $t = 0$ rotor 4 stops. The flight computer does not know this yet, so the healthy controller keeps flying. At $t = \tau$ (the *detection delay*) a detector raises the alarm and a supervisor switches to the failure-mode strategy, which must *capture* the drone into the relaxed hover from wherever it has drifted.

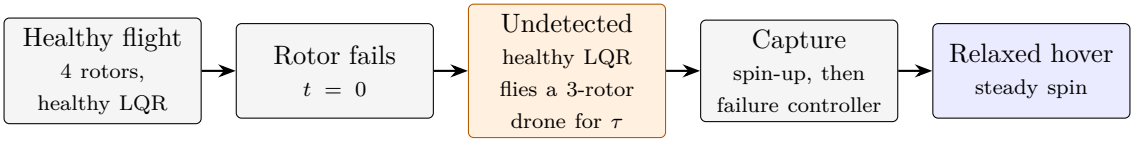


Figure 9: The failure timeline. RQ2 is about the orange phase: how long it may last.

10.2 Why the undetected phase is dangerous: a simple bound

Right after the failure the drone is still near level hover, so the body rates are small and the gyroscopic terms in (8) are negligible. The roll equation then reduces to

$$I_{xx}^{B}\,\dot{p} = l\,(f_2 - f_4) = l\,f_2, \tag{29}$$

because $f_4 = 0$. Rotor 2, opposite the failed rotor, now pushes one side up with nothing to balance it. The healthy controller notices the roll and reduces f_2, but it keeps its motors running, and a running motor cannot go below its idle thrust $f_{\min}$. Hence

$$\dot{p} \;\geq\; \frac{l\,f_{\min}}{I_{xx}^{B}} = \frac{0.17 \times 0.2}{3.2 \times 10^{-3}} = 10.6\ \text{rad/s}^2. \tag{30}$$

This holds for *any* healthy controller whose motor allocation keeps running motors at or above idle, as standard autopilots do, however it is tuned. Integrating twice gives a lower bound on the roll angle, $\frac{1}{2}(l f_{\min}/I_{xx}^{B})t^2$.

Figure 10 shows the simulation for three tunings of the healthy LQR ($\alpha = 0.3$, 1 and 3): the tilt reaches about 12 to 15° at 100 ms and 33 to 43° at 200 ms, so a more aggressive tuning helps only a little. Rotor 2's thrust falls close to its idle limit within about 100 ms; the controller then has no roll authority left.

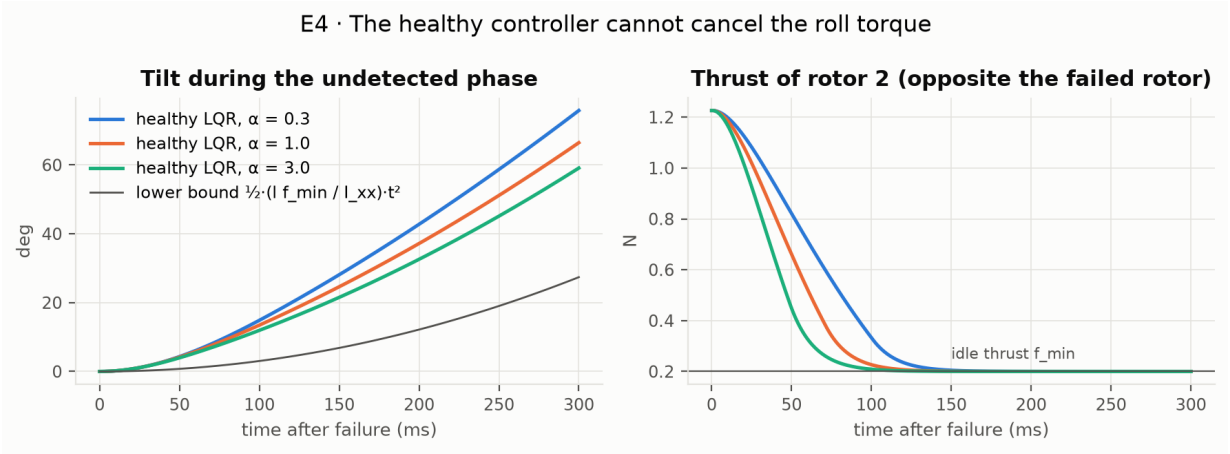


Figure 10: Undetected phase. Left: tilt for three healthy-controller tunings with the analytic lower bound. Right: thrust of rotor 2, which drops to idle.

10.3 Experimental design (E4)

Scenario. The drone hovers at height 0 on the nominal plant. At $t = 0$ rotor 4 fails. Until $t = \tau$ the healthy LQR flies (its command to rotor 4 has no effect). At $t = \tau$ a supervisor switches to one of these failure strategies:

- **Direct LQR**: the failure LQR is switched on at once.
- **Spin-up, then LQR** (the strategy of [1], used in their flight; the details below are read from their Fig. 4): rotors 1 and 3 run at a fixed thrust f_{spin}, rotor 2 is stopped, with no attitude feedback, until the rotation rate exceeds 10 rad/s; then the failure LQR takes over. We use $f_{\text{spin}} = 2.95$ N (read from the flight data) and, as a sensitivity case, $mg/2 = 2.45$ N.

At the switch, the failure controller's motor-thrust estimate is initialized from the supervisor's filter of the commands it sent before, never from the true thrust. Each run lasts 10 s, with the controller at 1 ms.

In the code: the detection-delay supervisor

```
def alive(self, t):                       # which rotors produce thrust
    return np.array([True, True, True, t < self.t_fail])

def __call__(self, t, x):
    if t < self.t_fail + self.tau:        # failure not yet detected
        u = self.healthy(t, x)            # (also updates a filter of the commands sent)
        return u
    if not self.switched:                 # first call after detection
        self.failure.reset(self.f_cmd_est)
        self.switched = True
    return self.failure(t, x)             # spin-up, then failure LQR
```

Success criterion, fixed before the runs. It is written in `metrics.py` and was not changed after seeing results. A run is

- **captured** if, over the last 1 s of a run that did not stop early, the reduced attitude is within 0.05 of $\bar{\boldsymbol{n}}$ and the body rates are within 2 rad/s of $\bar{\boldsymbol{\omega}}$;
- **safe** if it is captured, *and* the height loss (starting height minus lowest height) is below 2 m, *and* the drone never turns upside down ($n_z > 0$ throughout).

Monte Carlo. A real failure does not happen at a perfect hover. For each delay $\tau = 0, 5, 10, \ldots, 60$ ms and each spin-up thrust we ran 40 simulations from random pre-failure states (velocity with standard deviation 0.3 m/s, Euler angles 3°, body rates 0.3 rad/s; the same 40 seeds for every delay). The estimated probability of a safe recovery is $\hat{p} = k/n$ (k safe runs out of $n = 40$). Its 95% *Wilson* confidence interval is

$$c \pm h, \qquad c = \frac{\hat{p} + z^2/(2n)}{1 + z^2/n}, \qquad h = \frac{z}{1 + z^2/n}\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}, \qquad z = 1.96. \tag{31}$$

Unlike the textbook interval $\hat{p} \pm z\sqrt{\hat{p}(1-\hat{p})/n}$, it does not collapse to zero width when $\hat{p}$ is 0 or 1. We summarize each curve by τ_{90} and τ_{50}, the delays at which $\hat{p}$ first falls below 0.9 and 0.5 (linear interpolation between grid points). We also swept τ from 0 to 300 ms in 10 ms steps from the exact hover, for all three strategies, and for three healthy-controller tunings with the 2.95 N spin-up.

10.4 Results

Table 10: Spin-up then LQR: fraction of safe recoveries in 40 random starts per delay, with 95% Wilson intervals.

Delay τ	0 ms	5 ms	10 ms	15 ms	20 ms	30 ms	40 ms
2.95 N	1.00 [0.91, 1]	1.00 [0.91, 1]	0.97 [0.87, 1]	0.80 [0.65, 0.90]	0.42 [0.29, 0.58]	0.17 [0.09, 0.32]	0.07 [0.03, 0.20]
2.45 N	1.00 [0.91, 1]	0.95 [0.83, 0.99]	0.82 [0.68, 0.91]	0.45 [0.31, 0.60]	0.28 [0.16, 0.43]	0.07 [0.03, 0.20]	0.07 [0.03, 0.20]

Main result. With the flight spin-up thrust, the probability of a safe recovery falls below 0.9 at about **12 ms** of delay and below 0.5 at about **19 ms** (Table 10, Figure 11). With the lower thrust $mg/2$ these are 7 and 14 ms. If only attitude capture is required (ignoring height loss), the 50% point moves to 52 ms (40 ms with $mg/2$). Published detectors need about 20 to 130 ms [6–8], so with this strategy a safe recovery is unlikely within the time most current detectors need.

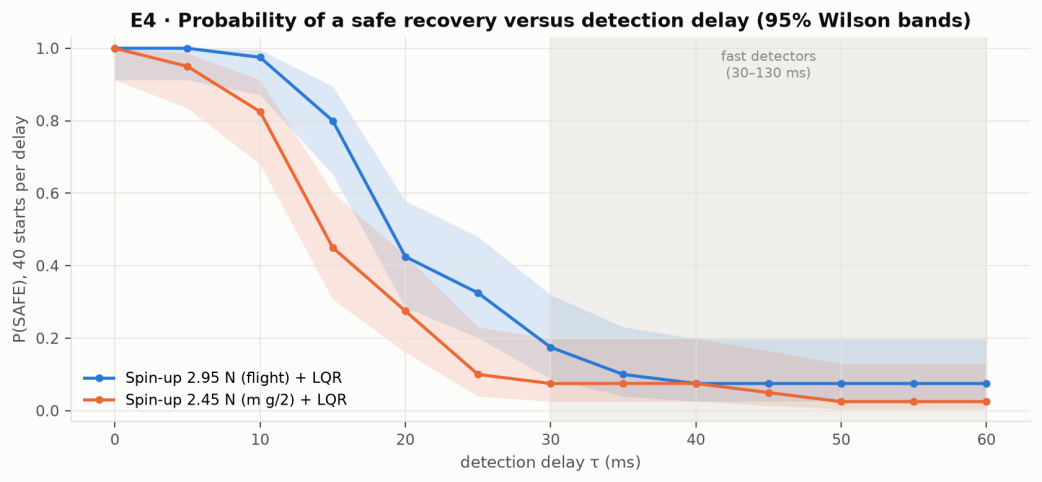


Figure 11: Probability of a safe recovery against detection delay for the two spin-up thrusts, with 95% Wilson bands. The shaded region marks the 30 to 130 ms reported for one fast detector [6].

No delay matches the flight. With the flight thrust and $\tau = 0$, the drone first climbs 0.40 m and then drops only 0.03 m below its starting height, qualitatively like the real flight, which climbed about 0.9 m.

Direct LQR fails. Switching the failure LQR on directly, without spin-up, captures the attitude in the end but always turns the drone upside down on the way and loses 9 to 16 m of height, at every delay (Figure 12). The LQR was designed for a drone that already spins at 19 rad/s; right after the failure the drone does not spin yet, and the linear model is far from valid there. Section 11 returns to this point.

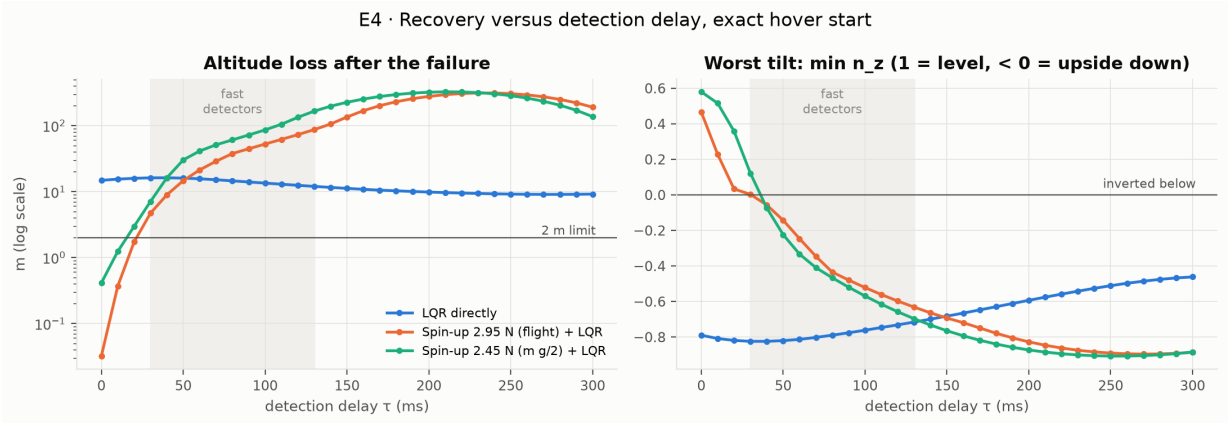


Figure 12: Exact hover start, delays 0 to 300 ms. Left: height loss (log scale) with the 2 m limit. Right: the worst tilt, $\min n_z$ (below 0 means upside down).

Tuning does not help. From the exact hover start, every delay up to 20 ms is safe for all three healthy-controller tunings ($\alpha = 0.3$, 1, 3), and 30 ms is not. This agrees with the idle-thrust bound: no tuning can stop the roll.

Scope. These numbers belong to this strategy and this model. In other simulation studies, nonlinear controllers recovered, with some loss of performance, at detection delays of about 0.1 to 0.2 s [8, 29], so the limit is not a property of rotor loss in general.

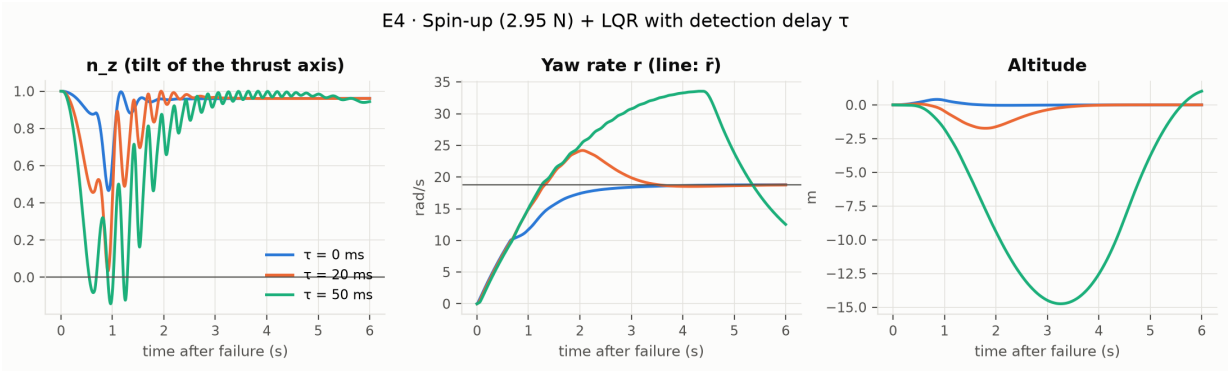


Figure 13: Spin-up (2.95 N) then LQR for delays of 0, 20 and 50 ms: tilt, yaw rate and height.

10.5 Do the MPCs tolerate more delay?

RQ2 asks how the capture controller changes the tolerable delay. As a first look we repeated the scenario on truth plant A with each controller of Section 9 as the capture controller (spin-up at 2.95 N, then LQR, analytical MPC or residual Koopman MPC). On a mismatched plant a controller designed on the nominal model settles at its own steady state, not at the plant's equilibrium, so capture was judged against each controller's own steady state (found by a 12 s run from the trim).

τ	LQR	MPC, analytical	MPC, residual Koopman
0 ms	safe (0.40 m)	safe (0.53 m)	safe (0.58 m)
10 ms	safe (1.69 m)	inverted (2.52 m)	inverted (2.80 m)
20 ms	inverted (6.56 m)	inverted (8.53 m)	inverted (7.94 m)

(Height loss in brackets; one run per entry.) LQR is safe up to 10 ms, both MPCs only without delay. We do not yet understand why. The LQR uses the same linear model and weights, so model validity alone does not explain the difference; candidates are the short 100 ms horizon and the thrust limits becoming active during capture, where the MPC and the clipped LQR act differently. (All three controllers run at 5 ms in this comparison.) This needs further study. Note that this table uses plant A and a 5 ms sample period for every controller (including the healthy one), while Table 10 uses the nominal plant and 1 ms; slower sampling alone adds effective delay, so their absolute numbers are not directly comparable.

11 A first mathematical certificate (RQ2)

Simulation can only test the starting states we try. A *certificate* is a mathematical proof, checked by a computer, that every state in some set recovers. In this report the certificates are proven for the *linearized* model with thrust limits; how well they carry over to the nonlinear drone is measured by simulation, not proven. RQ2 needs such a set: the *capture set* of the failure controller. If we can show that every state the drone can reach during the undetected phase lies inside the capture set, recovery is guaranteed. This section builds a first capture set (experiment E5) and measures how conservative it is.

11.1 The idea: an energy-like function that always decreases

Take the closed loop of the failure LQR on the 10-state linear model, written in deviations from the equilibrium. Without limits it would be $\dot{\boldsymbol{x}} = (A - BK)\boldsymbol{x}$. With the thrust limits it is

$$\dot{\boldsymbol{x}} = A\boldsymbol{x} + B\,\mathrm{sat}(F\boldsymbol{x}), \qquad F = -K, \tag{32}$$

where sat clips each input deviation to $[-\bar{u}_i, \bar{u}_i]$. Consider the quadratic function $V(\boldsymbol{x}) = \boldsymbol{x}^\top P \boldsymbol{x}$ with P positive definite, and the ellipsoid $\mathcal{E} = \{\boldsymbol{x} : V(\boldsymbol{x}) \leq 1\}$. If V decreases along every trajectory that starts in $\mathcal{E}$, no trajectory can leave $\mathcal{E}$, and all of them go to the equilibrium. Then $\mathcal{E}$ is a certified capture set. The question is how to find P, and how to handle the clipping, which makes the system nonlinear.

In plain words

Picture a bowl whose bottom is the equilibrium. If we can prove that the drone's state always moves downhill in the bowl, then any state that starts below a certain rim stays below it and slides to the bottom. The certificate is the bowl (P) and the rim (the ellipsoid). The thrust limits make the drone weaker when it is far from the bottom, so the rim cannot be arbitrarily high.

11.2 Handling the thrust limits (Hu and Lin)

Hu and Lin [41] give a sufficient condition based on the following observation. Choose a second, auxiliary gain H. If the state is in the region where $|H_i \boldsymbol{x}| \leq \bar{u}_i$ for every input i, then the clipped input $\mathrm{sat}(F\boldsymbol{x})$ is always some mixture (a convex combination) of the vectors

$$DF\boldsymbol{x} + (I - D)H\boldsymbol{x}, \qquad D \in \mathcal{D}, \tag{33}$$

where $\mathcal{D}$ is the set of diagonal matrices with 0 or 1 on the diagonal ($2^3 = 8$ of them for three inputs). Each D picks, input by input, either the real gain F or the auxiliary gain H. So if V decreases for each of these 8 *linear* systems, it also decreases for the clipped one, inside the region where $|H_i \boldsymbol{x}| \leq \bar{u}_i$.

11.3 Turning it into LMIs

For one vertex, "V decreases at rate at least $2\mu V$" means

$$\big(A + B(DF + D^- H)\big)^\top P + P\big(A + B(DF + D^- H)\big) + 2\mu P \preceq 0, \qquad D^- = I - D. \tag{34}$$

This contains the product of the unknowns P and H, so it is not linear. The standard fix is a change of variables: multiply from both sides by $Q = P^{-1}$ and write $Y = HQ$. The condition becomes

$$AQ + QA^\top + B(DFQ + D^- Y) + (DFQ + D^- Y)^\top B^\top + 2\mu Q \preceq -\epsilon I \qquad \forall D \in \mathcal{D}, \tag{35}$$

which is linear in the unknowns Q and Y: a linear matrix inequality (we add a small margin ϵ for numerical safety). The ellipsoid must also lie inside the region where the trick is valid. The largest value of $|H_i \boldsymbol{x}|$ on $\mathcal{E}$ is $\sqrt{H_i Q H_i^\top}$, so we need $H_i Q H_i^\top \leq \bar{u}_i^2$, which by a Schur complement is the LMI

$$\begin{pmatrix} \bar{u}_i^2 & Y_i \\ Y_i^\top & Q \end{pmatrix} \succeq 0, \qquad i = 1, 2, 3. \tag{36}$$

(Y_i is row i of Y.) Finally, we want the ellipsoid to be large in the direction we care about: from the equilibrium towards $\boldsymbol{x}_{\mathrm{on}}$, the state right after a failure in hover (zero body rates, level attitude, motors at $mg/4$). "$\beta \boldsymbol{x}_{\mathrm{on}}$ lies in $\mathcal{E}$" means $\beta^2\, \boldsymbol{x}_{\mathrm{on}}^\top Q^{-1} \boldsymbol{x}_{\mathrm{on}} \leq 1$; with $t = 1/\beta^2$ this is again a Schur complement,

$$\begin{pmatrix} t & \boldsymbol{x}_{\mathrm{on}}^\top \\ \boldsymbol{x}_{\mathrm{on}} & Q \end{pmatrix} \succeq 0, \tag{37}$$

and we minimize t, which maximizes β. A value $\beta \geq 1$ would mean the failure state itself is certified. (To get a "general-purpose" ellipsoid instead, we maximize its volume, $\log\det Q$, under the same constraints.)

Two practical details. The plant's thrust limits are not symmetric around the trim (rotor 2 is at 1.02 N, between 0.2 and 3.8 N), but the trick needs symmetric levels, so we take the smaller distance to either limit: $\bar{\boldsymbol{u}} = (1.76, 0.82, 1.76)$ N; a certificate with these levels also holds for the real limits. And the motor-state deviations must stay within their physical range, so we add $Q_{ii} \le \bar{u}_i^2$ for the three motor states. Without this bound the problem is badly conditioned: the open-loop model is only marginally stable, and the solver "certifies" an almost unbounded, physically meaningless set (tilt deviations above 1, motor deviations above 100 N).

In the code: the certificate in CVXPY

```
Q = cp.Variable((n, n), symmetric=True);  Y = cp.Variable((m, n))
cons = [Q >> eps * np.eye(n)]
for bits in itertools.product([0, 1], repeat=m):          # the 2^m vertices D
    D = np.diag(bits); Dm = np.eye(m) - D
    M = D @ F @ Q + Dm @ Y
    L = A @ Q + Q @ A.T + B @ M + (B @ M).T + 2 * decay * Q
    cons.append(0.5 * (L + L.T) << -eps * np.eye(n))     # LMI (lmi1)
for i in range(m):
    row = cp.reshape(Y[i, :], (1, n), order="C")
    cons.append(cp.bmat([[np.array([[ubar[i] ** 2]]), row], [row.T, Q]]) >> 0)  # (lmi2)
for i, b in enumerate(box):                               # motor states in range
    if np.isfinite(b): cons.append(Q[i, i] <= b ** 2)
t = cp.Variable()
cons.append(cp.bmat([[cp.reshape(t, (1, 1), order="C"), d.T], [d, Q]]) >> 0)
cp.Problem(cp.Minimize(t), cons).solve(solver="CLARABEL")  # beta = 1/sqrt(t)
```

The problems are modelled with CVXPY [46] and solved with the interior-point solver Clarabel [48], with SCS [49] as a fallback. A test samples 500 points on the boundary of a computed ellipsoid and checks that V really decreases there with the true clipping.

11.4 Results

Table 11: Certified ellipsoids for several required decay rates μ. The half-widths are those of the largest-volume ellipsoid (maximizing $\log\det Q$); β comes from a second problem that maximizes the extent towards the failure state. *Solver status "optimal_inaccurate" (solved to lower accuracy).

μ (1/s)	$\Delta p, \Delta q, \Delta r$ (rad/s)	$\Delta n_x, \Delta n_y$	Δz (m)	β
0.0*	2.23, 3.49, 6.20	0.42, 0.36	3.17	0.223
0.1	2.17, 3.49, 6.09	0.41, 0.35	2.99	0.221
0.3	2.07, 3.50, 5.89	0.40, 0.34	2.61	0.218
0.5	1.96, 3.52, 5.76	0.38, 0.32	2.17	0.215

Certified versus simulated. The certificate covers $\beta \approx 0.22$ of the way from the equilibrium to the failure state (Table 11); asking for a faster decay shrinks it only slightly. To see how conservative this is, we started the nonlinear simulation at $\beta \boldsymbol{x}_{\text{on}}$ for $\beta = 0.1, 0.2, \ldots, 1.0$ with the failure LQR and no delay. Every start up to $\beta = 0.7$ recovered without turning over (height loss 0.42 m at 0.7); at 0.8 and 0.9 the attitude was captured but the drone turned over, and at 1.0 it was not captured. So the certificate is about three times conservative in this direction, and this LQR on its own cannot recover safely from the failure state itself, which is why the spin-up phase is needed.

Where the controller fails. We also simulated a 21×21 grid of starting points in the plane of yaw-rate deviation Δr and tilt deviation Δn_y, all other states at equilibrium (Figure 14). 376 of 441 points recovered without turning over; 7% of these lie inside the certified slice, the rest is

the conservatism gap. All failures occur at spin rates of 4.8 rad/s or less, and none at 6.8 rad/s or more. In other words, the LQR works well as long as the drone already spins reasonably fast, and fails when it starts from a slow spin, which is exactly the situation right after a failure. A certificate built from one linearization at the full spin (19 rad/s) cannot reach that region.

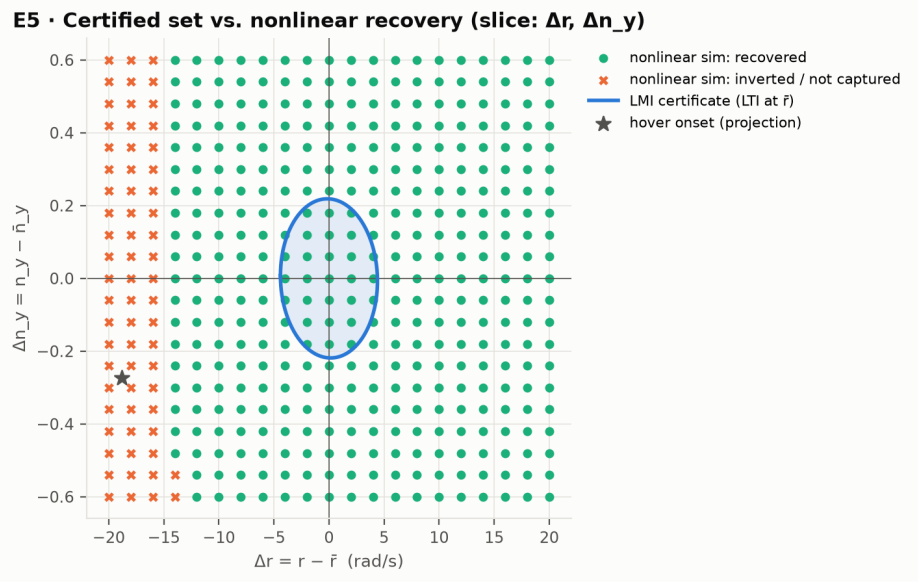


Figure 14: Slice $(\Delta r, \Delta n_y)$ of the certified ellipsoid ($\mu = 0.1\,\mathrm{s}^{-1}$) against a grid of nonlinear simulations. Dots: recovered; crosses: turned over or not captured; star: projection of the failure state (labelled "hover onset" in the plot). "LTI at $\bar{r}$" means the certificate uses the linear model at the equilibrium yaw rate.

11.5 From here to a certified delay budget (planned)

The results point to the next steps, which are not implemented yet:

1. **A certificate over the whole spin-up.** If the other operating-point values are held fixed, the linearized matrix depends affinely on the yaw rate r (otherwise they become additional scheduling parameters, and linearizing away from an equilibrium adds a drift term). Treating r as a parameter in an interval $[r_{\text{onset}}, \bar{r} + \delta]$ and requiring the LMIs (35) at the two ends with a common Q (a linear parameter-varying, LPV, certificate) would certify the whole interval. LPV synthesis on the yaw rate [19] and recoverable sets at the relaxed hover [27] exist; our aim is to use such a set as the capture target of a delay budget.

2. **The reachable set of the undetected phase.** During the delay the drone obeys an *affine* linear model (linear plus a constant term) $\dot{\boldsymbol{x}} = A_f \boldsymbol{x} + \boldsymbol{d}$ (healthy controller, rotor 4 missing; $\boldsymbol{d}$ is the constant push from the missing thrust). Starting from an ellipsoid $\mathcal{X}_0$ of pre-failure states, the states reachable at time τ form the shifted ellipsoid $\mathcal{R}(\tau) = e^{A_f \tau}\mathcal{X}_0 \oplus \{\int_0^\tau e^{A_f s}\boldsymbol{d}\, ds\}$, where $\oplus$ (the Minkowski sum) adds the fixed shift to every point of the set.

3. **The certified delay.** $\tau^\star = \sup\{\tau : M\mathcal{R}(\tau) \subseteq \mathcal{C}\}$, where M maps healthy-flight states to failure-controller coordinates and $\mathcal{C}$ is the capture set. Containment of one ellipsoid in another can be written as an LMI by the S-procedure (a standard way to turn "one quadratic inequality implies another" into a matrix inequality), and $\tau^\star$ is found by bisection.

The main risk is that a single quadratic function over the whole spin-up is infeasible or too conservative; the fallbacks are a parameter-dependent Lyapunov function, or a chain of nested ellipsoids along the spin-up.

12 Discussion, limitations and next steps

12.1 Status of the hypotheses

Table 12: Status of the hypotheses of Section 1.

	Hypothesis	Status
H1	Full-state EDMDc on the failed drone is unstable and loses to "no change" at long horizons	Supported (4 of 55 eigenvalues slightly outside the unit circle)
H2	The residual model predicts best	Supported from 100 ms to 1 s, on the training plant only (beyond about 2 s "no change" is better)
H3	Residual Koopman MPC removes the steady bias under mismatch	Partly: reduced by 32% on the training plant; worse on one held-out variant
H4	The residual model needs less data	Supported (10 episodes beat 40)
H5	The linear certificate does not contain the failure state; one over the spin-up does	First half supported ($\beta = 0.22$); second half not yet tested
H6	The certified $\tau^\star$ is finite and of the order of real detector delays	Not yet tested; the empirical limit (12 to 19 ms) is shorter than detectors need
H7	The certified $\tau^\star$ is within 2 to 3 times the empirical value	Not yet tested

12.2 What the results mean

Physics first, learning second. The physics model of Mueller and D'Andrea, which we reproduced to every printed digit except in one internally inconsistent published case, already describes the spinning drone well: it gives the right equilibrium, the right flight spin-up time and a controller that works. Learning helps only in the second order, and only when the learned part is restricted to the error. Learning everything from data produced an unstable model, as our proposal predicted.

A calibration, not yet a general model. The learned residual is accurate on the plant it saw and harmful on a different plant. As it stands it behaves like an offline calibration of one vehicle. Online adaptation, or a residual that also uses a short history of states, is needed before it can be called a general correction for rotor failure.

Detection time is the bottleneck. The most striking result is how little detection delay the published strategy tolerates in our simulation: around 10 to 20 ms, while published detectors need about 20 to 130 ms. The cause is simple and robust: the healthy controller cannot stop the roll because the opposite rotor cannot go below idle thrust. This makes a certified delay budget, and controllers that tolerate longer delays, practically important.

12.3 Limitations

1. **Spin-up thrust.** The delay tolerance depends on the spin-up thrust (12 against 7 ms for the 90% level). The flight value is read from a plot and carries some uncertainty.
2. **Nominal plant for the delay study.** The delay study uses the nominal plant; the published models over-predict the steady spin rate in flight by about 20 to 27%.
3. **Generalization.** The residual model helps reliably only on the plant family it was trained on.

4. **Linear certificates.** The certificate holds for the linearized loop with saturation. The gap to the nonlinear system is measured by simulation, not proven.
5. **Truth-plant realism.** The truth plants add thrust loss, inertia error and motor lag, but not yet air-relative thrust or quadratic drag, both of which we validated in our implementation of the IJRR model. There is no sensor noise and no translational drag yet.
6. **Certificate for the learned controller.** The clipping level $\boldsymbol{d}_{\max}$ is a percentile of the training residuals, not a bound on the true mismatch, and a constrained MPC needs more than the saturation conditions of Section 11. Both must be addressed before the residual Koopman loop can be certified.
7. **Simulation only.** No result has been tested on hardware.
8. **Full-state measurement.** All controllers are given the exact state. A real spinning drone needs an estimator for the tilt, height and vertical speed, and sensor noise will affect both the learned residual and the MPC.
9. **Closed-loop data.** The learning data are recorded under LQR feedback with added excitation, which correlates states and inputs; possible bias from this is not analysed.
10. **Missing baselines.** An offset-free (integral-action) MPC is the obvious classical alternative for removing a steady error, and grey-box estimation of the few uncertain physical parameters (thrust loss, inertias, motor time constant) is the obvious model-based alternative to a black-box residual. Neither has been compared yet.
11. **Literature access.** Several closely related papers [18, 19, 25, 27] have not yet been read in full; our statements about them rely on abstracts, a thesis and citing papers.

12.4 Plan for the remaining weeks

The course has a final progress presentation on 31 October 2026, and the journal-style final report is due on 13 November 2026.

Dates	Work
Until 18 Oct	LPV model in the yaw rate and a saturation-aware certificate over the spin-up. MPC on the full-state Koopman model (the second controller of our proposal) and an integral-action MPC baseline. Read the remaining key papers in full.
19 to 25 Oct	Reachable set of the undetected phase and the containment test, giving a first certified $\tau^\star$; comparison with the empirical curve. Online adaptation of the residual by recursive least squares.
26 to 31 Oct	Sensitivity of $\tau^\star$ to the spin-up thrust, idle thrust and motor lag; air-relative thrust in the truth plant. Final progress presentation on 31 Oct.
1 to 13 Nov	Bound on the true model mismatch and a certificate for the residual Koopman loop if time allows; final simulations and the journal-style report, due 13 Nov.

12.5 Conclusion

We built a simulator of a quadrotor with complete rotor loss and showed that it reproduces three published papers and the spin-up time of a real flight. On the plant it was trained on, a Koopman model that learns only the error of the physics model is the most accurate predictor from 100 ms to 1 s, needs less data than a Koopman model of the full dynamics, and reduces a steady altitude error by one third inside a convex MPC; it does not yet carry over to a plant with different parameters. In our simulation, the published recovery strategy recovers safely with high probability only for detection delays of about 10 to 20 ms, shorter than most published detectors need (about 20 to 130 ms), because nothing can stop the roll while the failure is undetected. A

first LMI certificate covers 22% of the way to the failure state, and the controller fails only at low spin rates, which points to a certificate over the spin-up as the next step. The remaining weeks build that certificate and the certified delay budget, and evaluate each capture controller, including the residual Koopman MPC, by the delay it tolerates.

A Glossary

Terms in alphabetical order. The section where each is first used in detail is given in brackets.

Term	Meaning
Actuator	A device that acts on the system; here the four motor–propeller units.
Affine	Linear plus a constant term, $A\boldsymbol{x} + \boldsymbol{d}$.
Air-relative thrust	Propeller thrust computed from the propeller's speed relative to the air ($\Omega_i + r$) rather than relative to the body; matters when the body spins fast (6.2.2).
Algebraic Riccati equation	The matrix equation whose solution P gives the optimal LQR gain; "discrete" version (DARE) for sampled systems (2, 9).
Attainable moment set	The set of torques the remaining rotors can produce within their thrust limits.
Body frame	Coordinate frame fixed to the drone (4).
Body rates (p, q, r)	Rotation rates about the body x, y, z axes: roll rate, pitch rate, yaw rate.
Brent's method	A reliable one-dimensional root finder that needs a bracket where the function changes sign (4).
Bryson's rule	Choosing LQR weights as $1/(\text{largest acceptable value})^2$ (7).
Capture	Bringing the drone into the relaxed hover after a failure; a "capture controller" does this (10).
Capture set	A set of states from which the capture controller is proven to recover (11).
Central differences	Numerical derivative $(f(x+\epsilon) - f(x-\epsilon))/2\epsilon$ (2).
Certificate	A computer-checked mathematical proof that a property (here, recovery) holds for a whole set of states (11).
Closed loop	The system together with its feedback controller.
Controllability	Whether the inputs can steer the state anywhere; tested by the rank of $[B, AB, \dots]$ or the PBH test (7).
Convex problem	An optimization problem in which every local minimum is a global one, so solvers find the best solution reliably; QPs with positive semidefinite cost and LMI problems are convex. With a strictly convex cost the solution is unique.
CVXPY	A Python library for writing convex optimization problems; it passes them to solvers such as OSQP or Clarabel.
Damping ratio ζ	How quickly an oscillation dies out; 0 means never, 1 means no overshoot.
Delay margin	The largest pure input delay a feedback loop tolerates before becoming unstable (7).
Detection delay τ	Time between the rotor failure and the moment the flight computer knows about it (10).
Dictionary	The list of functions (observables) used to lift the state in EDMD (8).
EDMD / EDMDc	Extended dynamic mode decomposition (with control): fitting a linear model on a dictionary of functions of the state by least squares (2).
Eigenvalue	A number λ with $A\boldsymbol{v} = \lambda\boldsymbol{v}$; describes one mode of a linear system (2).
Equilibrium (trim)	A state and input at which nothing changes (2).
Euler angles	Roll ϕ, pitch θ, yaw ψ: three angles that describe orientation; break down at some orientations (gimbal lock).
Excitation	Random changes added to the inputs during data collection so that the data show how the system responds (8).
FDI / FDD	Fault detection and isolation / diagnosis: algorithms that notice and locate a failure.
Feedback linearization	A nonlinear control method that cancels the nonlinear terms by the control input so the remaining loop is linear.

Term	Meaning
Floquet theory	Stability theory for linear systems whose coefficients repeat periodically in time (4).
FTC	Fault-tolerant control: control that keeps working after a failure. Active FTC switches controller after detection; passive FTC does not need detection.
Gramian	A matrix measuring how strongly each direction of the state can be reached (controllability) or seen (observability) in a given time (7).
Gyroscopic effect	The torque needed to change the direction of an angular momentum; makes spinning bodies precess (4).
H_∞ loop shaping	A robust linear control design that shapes the loop's frequency response and maximizes a robustness margin.
Healthy controller	The normal controller used with four working rotors.
Horizon	How far ahead MPC predicts (here 20 steps, 100 ms).
INDI	Incremental nonlinear dynamic inversion: a nonlinear control method that uses measured accelerations.
Inertia matrix	How hard it is to change the rotation of a body about each axis.
Interior-point method	A family of algorithms for convex optimization that moves through the inside of the feasible set; Clarabel is one.
Invariant set	A set that no trajectory starting inside it can leave (11).
Jacobian	Matrix of partial derivatives; the linearization of a nonlinear function (2).
Koopman operator	A linear operator that advances functions of the state in time; lets nonlinear systems be described linearly in a lifted space (2).
Lifted state	The vector of dictionary functions $\boldsymbol{z} = \psi(\boldsymbol{x})$.
LMI	Linear matrix inequality: a convex constraint saying a matrix, linear in the unknowns, is positive or negative semidefinite (11).
LPV	Linear parameter-varying: a linear model whose matrices depend on a measured, changing parameter (here the yaw rate).
LQR	Linear-quadratic regulator: the state-feedback gain minimizing a quadratic cost (2).
LTI	Linear time-invariant: a model $\dot{\boldsymbol{x}} = A\boldsymbol{x} + B\boldsymbol{u}$ with constant matrices.
Lyapunov function	An energy-like function that decreases along the motion; its existence proves stability (11).
Measured disturbance	A known, additive term in an MPC prediction model; used here for the learned residual (9).
Minkowski sum $\oplus$	The set of all sums $a + b$ with a from one set and b from another (11).
Monte Carlo	Repeating a simulation from many random starts to estimate probabilities (10).
Motor lag	Delay of the motor thrust behind its command, modelled as a first-order lag with time constant σ (4).
MPC	Model predictive control: repeatedly optimize the inputs over a short future horizon and apply the first one (2, 9).
NMPC	Nonlinear MPC: MPC with a nonlinear prediction model (no longer a QP).
Normalized RMSE	Root mean square error after dividing each state by its typical size (8).
Observability	Whether the state can be worked out from the sensor outputs (7).
Offset-free MPC	MPC that estimates a constant disturbance and compensates it, removing steady errors like integral action.
Open loop	Without feedback; an open-loop prediction uses no later measurements.
Operator splitting	A way of solving an optimization problem by alternating between simpler subproblems; used by OSQP.
OSQP	A fast solver for quadratic programs.
Overfitting	Fitting the noise or quirks of the training data instead of the true pattern.
PBH test	Popov–Belevitch–Hautus test: checks controllability (or observability) mode by mode (7).
Plant	The system being controlled (here the drone, or a truth plant in simulation).
Positive definite	A symmetric matrix P with $\boldsymbol{x}^\top P\boldsymbol{x} > 0$ for all $\boldsymbol{x} \neq 0$, written $P \succ 0$; $\succeq$ allows equality (2).

Term	Meaning
QP	Quadratic program: minimize a quadratic cost subject to linear constraints; convex.
Quaternion	Four numbers that describe an orientation without gimbal lock (4).
Randomized plants	Training on many plants with randomly drawn parameters, also called domain randomization (8).
Reaction torque	Twisting torque on the body from a spinning propeller (1).
Reduced attitude $\boldsymbol{n}$	The inertial "up" direction seen in body axes, $R^\top \boldsymbol{e}_3$; all that is controlled after the failure (4).
Relaxed hover	The spinning equilibrium of a multicopter that has lost rotors (4).
Residual	The difference between the truth and the physics model's prediction (8).
Ridge regression	Least squares with a penalty on the size of the coefficients (2).
RK4	The classical fourth-order Runge–Kutta integration method (2).
Rotation matrix R	Matrix turning body-frame vectors into inertial-frame vectors.
Saturation	Clipping of an input at its physical limits (2).
Schur complement	A rule that turns certain nonlinear matrix inequalities into LMIs (11).
SE(3)	The set of all positions and orientations of a rigid body.
Singular values	Numbers that measure how much a matrix stretches vectors in each direction; a value near zero means a direction the matrix almost loses.
Sliding-mode control	A robust nonlinear control method that forces the state onto a chosen surface by switching control.
Spectral radius	The largest eigenvalue magnitude of a discrete-time matrix; above 1 means growth.
Spin-up	Flying on two opposing rotors with no attitude feedback until the drone spins fast enough for the failure LQR (10).
S-procedure	A standard technique that turns "one quadratic inequality implies another" into an LMI (11).
Standardize	Subtract the mean and divide by the standard deviation.
State	The smallest set of numbers that determines the future of a system given its inputs (2).
Sum-of-squares (SOS) programming	An extension of LMI methods that certifies polynomial inequalities, used for nonlinear stability regions.
Supervisor	The logic that switches from the healthy to the failure controller after detection (10).
Switched system	A system that jumps between several models, for example healthy and faulty; "dwell time" is the minimum average time between switches for which stability is guaranteed.
Thrust	The upward force of a propeller, $f = \kappa_f \Omega^2$.
Truth plant	A simulated plant with effects the nominal model lacks, used to generate data and to test controllers (4.10).
Wilson interval	A confidence interval for a probability estimated from a count (10).
Zero-order hold (ZOH)	Holding a digital command constant until the next sample (2).

B Symbols

Symbol	Meaning	Value / unit
m	mass	$0.50\,\mathrm{kg}$
g	gravity	$9.81\,\mathrm{m/s^2}$
l	arm length	$0.17\,\mathrm{m}$
I^B, I^T, I^P	inertia of body, body with propellers, one propeller	$\mathrm{kg\,m^2}$
κ_f	thrust coefficient, $f = \kappa_f \Omega^2$	$6.41 \times 10^{-6}\,\mathrm{N\,s^2/rad^2}$
κ_τ	reaction torque per unit thrust	$0.0169\,\mathrm{m}$
γ	linear yaw-drag coefficient	$2.75 \times 10^{-3}\,\mathrm{N\,m\,s/rad}$

Symbol	Meaning	Value / unit
f_i, f_Σ	thrust of rotor i, total thrust	N
$f_{\min}$, $f_{\max}$	running-motor thrust limits	0.2, 3.8 N
σ	motor time constant	15 ms
u_i	thrust command of rotor i	N
Ω_i, Ω_Σ	signed speed of propeller i, and their sum	rad/s
$\boldsymbol{d}$, $\boldsymbol{v}$	position and velocity (inertial frame)	m, m/s
R, $\boldsymbol{q}$	rotation matrix and quaternion (body to inertial)	
$\boldsymbol{\omega} = (p, q, r)$	body rates	rad/s
$\boldsymbol{n} = R^\top \boldsymbol{e}_3$	reduced attitude	
$\boldsymbol{e}_3$	$(0, 0, 1)$	
η, k	thrust efficiency $1 - k\|\boldsymbol{\omega}\|^2$ and its coefficient	$\mathrm{s}^2/\mathrm{rad}^2$
ρ	thrust ratio f_2/f_1 at the relaxed hover	0.5
$\bar{(\cdot)}$	value at the equilibrium	
$\bar{a}$	gyroscopic coupling constant	-10.10 rad/s
A, B	continuous-time linear model	
A_d, B_d	discrete-time (ZOH) model	
T_s	sample period of learning and MPC	5 ms
K, $F = -K$	feedback gain	
Q, R	LQR / MPC weights (Section 11: $Q = P^{-1}$)	
P, P_N	Riccati solution / Lyapunov matrix; MPC terminal cost	
N	MPC horizon	20 steps
$\hat{\boldsymbol{f}}$	filtered-command estimate of the motor thrusts	N
$\boldsymbol{\xi}$, ψ, $\boldsymbol{\zeta}$	learning features, dictionary, lifted state	
W, $\boldsymbol{d}_{\max}$	residual regression matrix, clipping level	
τ, $\tau^\star$	detection delay, certified maximum delay	ms
$\bar{\boldsymbol{u}}$	symmetric saturation levels	(1.76, 0.82, 1.76) N
μ	required decay rate in the certificate	1/s
β	certified fraction of the way to the failure state	
α	aggressiveness factor of the healthy LQR	

C Software and reproducibility

The simulator, analysis and experiments are written in Python with NumPy [44], SciPy [45] and Matplotlib [50]. Convex problems are modelled with CVXPY [46] and solved with Clarabel [48], SCS [49] and OSQP [47]; tests use pytest [51]. Package versions are pinned in `requirements.txt`. The code is available at `https://github.com/pranshuk22/quadrotor-rotor-loss-recovery`.

```
python3 -m venv .venv && .venv/bin/pip install -r requirements.txt
.venv/bin/python -m pytest -q        # 46 pass + 1 expected failure
.venv/bin/python experiments/v0_validation_report.py
.venv/bin/python experiments/e1_equilibrium.py   # and e2 ... e7
```

Each script writes a table to `results/tables/` and figures to `results/figures/`; all random seeds are fixed in the scripts (for example seeds 11, 12, 13 and 21 for the training, test and randomized data of E6). Runtimes are for an 8-core laptop.

References

[1] Mark W. Mueller and Raffaello D'Andrea. Stability and control of a quadrocopter despite the complete loss of one, two, or three propellers. In *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, pages 45–52, 2014. doi: 10.1109/ICRA.2014.6906588.

ID	Script	Question	Runtime
V0	`v0_validation_report.py`	Does the model match published numbers and flight data?	seconds
E1	`e1_equilibrium.py`	Relaxed hover against the paper; ρ sweep	seconds
E2	`e2_structure.py`	Eigenvalues, controllability, observability by sensor set	seconds
E3	`e3_lqr.py`	LQR designs, delay margins, nonlinear checks	about 15 s
E4	`e4_delay_sweep.py`	Effect of detection delay (Monte Carlo)	several minutes
E5	`e5_first_certificate.py`	LMI certificate and its conservatism	about 8 min
E6	`e6_koopman_prediction.py`	Physics vs full-state vs residual EDMDc	about 2.5 min
E7	`e7_mpc_closed_loop.py`	LQR vs analytical MPC vs residual Koopman MPC	about 3 min

[2] Mark W. Mueller and Raffaello D'Andrea. Relaxed hover solutions for multicopters: Application to algorithmic redundancy and novel vehicles. *The International Journal of Robotics Research*, 35(8):873–889, 2016. doi: 10.1177/0278364915596233.

[3] Edoardo Todde, Simone Martini, Alessandro Rizzo, and Kimon P. Valavanis. Residual Koopman-based model predictive control of quadrotors. In *Proc. International Conference on Unmanned Aircraft Systems (ICUAS)*, pages 50–56, 2026. doi: 10.1109/ICUAS69441.2026.11598621.

[4] D. Bruder, D. Bombara, and R. J. Wood. A Koopman-based residual modeling approach for the control of a soft robot arm. *The International Journal of Robotics Research*, 44(3):388–406, 2025. doi: 10.1177/02783649241272114.

[5] Sriram S. K. S. Narayanan and Umesh Vaidya. RK-MPC: Residual Koopman model predictive control for quadruped locomotion in offroad environments, 2026. arXiv:2604.04221.

[6] Bram Strack van Schijndel, Sihao Sun, and Coen de Visser. Fast loss of effectiveness detection on a quadrotor using onboard sensors and a Kalman estimation approach. In *Proc. International Conference on Unmanned Aircraft Systems (ICUAS)*, pages 1–8, 2023. doi: 10.1109/ICUAS57906.2023.10156464. arXiv:2102.06439.

[7] Chengyu Yang, Jiahao Yu, Sheng Cheng, Ziyin Han, Junjie Gao, Bihao Mo, and Naira Hovakimyan. Failure detection and recovery for quadrotors in the presence of severe rotor failures with multiple model $\mathcal{L}_1$ adaptive controller. *IEEE Robotics and Automation Letters*, 11(1):346–353, 2026. doi: 10.1109/LRA.2025.3630964.

[8] Xiaobin Zhou, Miao Wang, Chengao Li, Can Cui, Ruibin Zhang, Yongchao Wang, Chao Xu, and Fei Gao. Rotor-failure-aware quadrotors flight in unknown environments. *IEEE Transactions on Aerospace and Electronic Systems*, 62:6371–6383, 2026. doi: 10.1109/TAES.2026.3664067. arXiv:2510.11306.

[9] Ashutosh Simha, Sharvaree Vadgama, and Soumyendu Raha. A geometric approach to rotor failure tolerant trajectory tracking control design for a quadrotor, 2017. arXiv:1704.00327.

[10] Mohamed Al Lawati and Adeel Akhtar. Quasi-static fault-tolerant feedback control of a quadrotor under rotor failure with provable safety guarantees. In *Proc. IEEE Conference on Control Technology and Applications (CCTA)*, pages 681–686, 2026. doi: 10.1109/CCTA62090.2026.11684234. arXiv:2609.23918.

[11] Damiàn Rotondo, Jean-Christophe Ponsart, Didier Theilliol, Fatiha Nejjari, and Vicenç Puig. A virtual actuator approach for the fault tolerant control of unstable linear systems subject to actuator saturation and fault isolation delay. *Annual Reviews in Control*, 39:68–80, 2015. doi: 10.1016/j.arcontrol.2015.03.006.

[12] Damiàn Rotondo, Jean-Christophe Ponsart, Fatiha Nejjari, Didier Theilliol, and Vicenç Puig. Virtual actuator-based FTC for LPV systems with saturating actuators and FDI delays. In *Proc. Conference on Control and Fault-Tolerant Systems (SysTol)*, pages 831–837, 2016. doi: 10.1109/SYSTOL.2016.7739850.

[13] Milan Korda and Igor Mezić. Linear predictors for nonlinear dynamical systems: Koopman operator meets model predictive control. *Automatica*, 93:149–160, 2018. doi: 10.1016/j.automatica.2018.03.046.

[14] Matthew O. Williams, Ioannis G. Kevrekidis, and Clarence W. Rowley. A data-driven approximation of the Koopman operator: Extending dynamic mode decomposition. *Journal of Nonlinear Science*, 25(6):1307–1346, 2015. doi: 10.1007/s00332-015-9258-5.

[15] Stephen Boyd, Laurent El Ghaoui, Eric Feron, and Venkataramanan Balakrishnan. *Linear Matrix Inequalities in System and Control Theory*, volume 15 of *SIAM Studies in Applied Mathematics*. SIAM, Philadelphia, 1994. doi: 10.1137/1.9781611970777.

[16] Sihao Sun, Xuerui Wang, Qiping Chu, and Coen de Visser. Incremental nonlinear fault-tolerant control of a quadrotor with complete loss of two opposing rotors. *IEEE Transactions on Robotics*, 37(1):116–130, 2021. doi: 10.1109/TRO.2020.3010626. arXiv:2002.07837.

[17] Alessandro Freddi, Alexander Lanzon, and Sauro Longhi. A feedback linearization approach to fault tolerance in quadrotor vehicles. In *Proc. 18th IFAC World Congress (IFAC Proceedings Volumes 44(1))*, pages 5413–5418, 2011. doi: 10.3182/20110828-6-IT-1002.02016.

[18] Alexander Lanzon, Alessandro Freddi, and Sauro Longhi. Flight control of a quadrotor vehicle subsequent to a rotor failure. *Journal of Guidance, Control, and Dynamics*, 37(2):580–591, 2014. doi: 10.2514/1.59869.

[19] Johannes Stephan, Lorenz Schmitt, and Walter Fichter. Linear parameter-varying control for quadrotors in case of complete actuator loss. *Journal of Guidance, Control, and Dynamics*, 41(10):2232–2246, 2018. doi: 10.2514/1.G003441.

[20] Ahmed Khattab, Halim Alwi, and Christopher Edwards. Mitigating total rotor failure in quadrotor using LPV based sliding mode control scheme. In *Proc. 4th Conference on Control and Fault Tolerant Systems (SysTol)*, pages 98–103, 2019. doi: 10.1109/SYSTOL.2019.8864740.

[21] Sihao Sun, Matthias Baert, Bram Strack van Schijndel, and Coen de Visser. Upset recovery control for quadrotors subjected to a complete rotor failure from large initial disturbances. In *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, pages 4273–4279, 2020. doi: 10.1109/ICRA40945.2020.9197239. arXiv:2002.09425.

[22] Fang Nan, Sihao Sun, Philipp Foehn, and Davide Scaramuzza. Nonlinear MPC for quadrotor fault-tolerant control. *IEEE Robotics and Automation Letters*, 7(2):5047–5054, 2022. doi: 10.1109/LRA.2022.3154033. arXiv:2109.12886.

[23] Zhiwen Sun, Fubiao Zhang, and Shuaipeng Lang. Fault-tolerant model predictive control of a quadrotor with an unknown complete rotor failure. In *Proc. 41st Chinese Control Conference (CCC)*, pages 4087–4093, 2022. doi: 10.23919/CCC55666.2022.9901659.

[24] Chenxu Ke, Kai-Yuan Cai, and Quan Quan. Uniform passive fault-tolerant control of a quadcopter with one, two, or three rotor failure. *IEEE Transactions on Robotics*, 39(6):4297–4311, 2023. doi: 10.1109/TRO.2023.3297048.

[25] Hasan Basak and Emmanuel Prempain. Switching recovery control of a quadcopter UAV. In *Proc. European Control Conference (ECC)*, pages 3641–3646, 2015. doi: 10.1109/ECC.2015.7331096.

[26] Hasan Basak. *Robust switching recovery control of a quadcopter aerial vehicle model*. PhD thesis, University of Leicester, 2018. hdl:2381/41081.

[27] Xin Qi, Didier Theilliol, Yuqing He, and Jianda Han. Recoverable set computation for post-fault/failure quadrotors based on sum of squares (SOS). In *Proc. 3rd Conference on Control and Fault-Tolerant Systems (SysTol)*, pages 73–78, 2016. doi: 10.1109/SYSTOL.2016.7739731.

[28] Eeshan Kulkarni, Narasimhan Sundararajan, and Suresh Sundaram. Heli-quad design for full-attitude fault-tolerant control under complete failure of an actuator. *Unmanned Systems*, 12(6):1001–1022, 2024. doi: 10.1142/S2301385024500341. arXiv:2011.07529v3.

[29] Jiehao Chen, Kaidong Zhao, Zihan Liu, YanJie Li, and Yunjiang Lou. Learning-based passive fault-tolerant control of a quadrotor with rotor failure. In *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1876–1883, 2025. doi: 10.1109/IROS60139.2025.11245951. arXiv:2503.02649.

[30] Florin Stoican and Sorin Olaru. *Set-Theoretic Fault-Tolerant Control in Multisensor Systems*. Wiley-ISTE, 2013. doi: 10.1002/9781118649428.

[31] Jong-Yeob Shin and Christine M. Belcastro. Performance analysis on fault tolerant control system. *IEEE Transactions on Control Systems Technology*, 14(5):920–925, 2006. doi: 10.1109/TCST.2006.876911.

[32] Youngim Nam, Kangmin Lee, Hyo-Sang Shin, and Cheolhyeon Kwon. Fault tolerant motion planning for a quadrotor subject to complete rotor failure. *Aerospace Science and Technology*, 154:109529, 2024. doi: 10.1016/j.ast.2024.109529.

[33] Steven L. Brunton, Bingni W. Brunton, Joshua L. Proctor, and J. Nathan Kutz. Koopman invariant subspaces and finite linear representations of nonlinear dynamical systems for control. *PLoS ONE*, 11(2):e0150171, 2016. doi: 10.1371/journal.pone.0150171.

[34] S. S. Narayanan, D. Tellez-Castro, S. Sutavani, and U. Vaidya. SE(3) Koopman-MPC: Data-driven learning and control of quadrotor UAVs. *IFAC-PapersOnLine*, 56(3):607–612, 2023. doi: 10.1016/j.ifacol.2023.12.091.

[35] S. Martini, E. Todde, M. Stefanovic, M. J. Rutherford, A. Rizzo, and K. P. Valavanis. Koopman-based model predictive control of quadrotors. In *Proc. International Conference on Unmanned Aircraft Systems (ICUAS)*, pages 207–213, 2025. doi: 10.1109/ICUAS65942.2025.11007936.

[36] Jayden Dongwoo Lee, Hyeonmin Kim, Yoonseong Kim, and Hyochoong Bang. Koopman fault-tolerant model predictive control for multirotor with actuator fault. In *Proc. 24th International Conference on Control, Automation and Systems (ICCAS)*, pages 231–236, 2024. doi: 10.23919/ICCAS63016.2024.10773232.

[37] Jayden Dongwoo Lee, Sukjae Im, Lamsu Kim, Hyungjoo Ahn, and Hyochoong Bang. Data-driven fault detection and isolation for multirotor system using Koopman operator. *Journal of Intelligent & Robotic Systems*, 110(3):128, 2024. doi: 10.1007/s10846-024-02142-y.

[38] Yonghao Fu, Cheng Hu, Haokun Xiong, Zhanpeng Bao, Wenyuan Du, Edoardo Ghignone, Michele Magno, Lei Xie, and Hongye Su. Residual Koopman model predictive control for enhanced vehicle dynamics with small on-track data input, 2025. arXiv:2507.18396.

[39] Ao Jin, Qiujin Liang, Tao Zhang, Panfeng Huang, and Fan Zhang. Differentiable physics-informed adaptive Koopman control for stable flight under unknown disturbances, 2025. arXiv:2506.08319.

[40] Robin Strässer, Manuel Schaller, Karl Worthmann, Julian Berberich, and Frank Allgöwer. SafEDMD: A Koopman-based data-driven controller design framework for nonlinear dynamical systems. *Automatica*, 185:112732, 2026. doi: 10.1016/j.automatica.2025.112732. arXiv:2402.03145.

[41] Tingshu Hu and Zongli Lin. *Control Systems with Actuator Saturation: Analysis and Design*. Birkhäuser, Boston, 2001. doi: 10.1007/978-1-4612-0205-9.

[42] Sihao Sun, Giovanni Cioffi, Coen de Visser, and Davide Scaramuzza. Autonomous quadrotor flight despite rotor failure with onboard vision sensors: Frames vs. events. *IEEE Robotics and Automation Letters*, 6(2):580–587, 2021. doi: 10.1109/LRA.2020.3048875. arXiv:2102.13406.

[43] Prashant Solanki and Coen C. de Visser. Attitude estimation of a quadcopter with one fully damaged rotor using on-board MARG sensors. In *AIAA SciTech 2022 Forum*, 2022. doi: 10.2514/6.2022-0857.

[44] Charles R. Harris et al. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020. doi: 10.1038/s41586-020-2649-2.

[45] Pauli Virtanen et al. SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3):261–272, 2020. doi: 10.1038/s41592-019-0686-2.

[46] Steven Diamond and Stephen Boyd. CVXPY: A Python-embedded modeling language for convex optimization. *Journal of Machine Learning Research*, 17(83):1–5, 2016.

[47] Bartolomeo Stellato, Goran Banjac, Paul Goulart, Alberto Bemporad, and Stephen Boyd. OSQP: An operator splitting solver for quadratic programs. *Mathematical Programming Computation*, 12(4):637–672, 2020. doi: 10.1007/s12532-020-00179-2.

[48] Paul J. Goulart and Yuwen Chen. Clarabel: An interior-point solver for conic programs with quadratic objectives. *Mathematical Programming Computation*, 2026. doi: 10.1007/s12532-026-00320-7. arXiv:2405.12762.

[49] Brendan O'Donoghue, Eric Chu, Neal Parikh, and Stephen Boyd. Conic optimization via operator splitting and homogeneous self-dual embedding. *Journal of Optimization Theory and Applications*, 169(3):1042–1068, 2016. doi: 10.1007/s10957-016-0892-3.

[50] John D. Hunter. Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007. doi: 10.1109/MCSE.2007.55.

[51] Holger Krekel et al. pytest. `https://pytest.org`, 2026. Version 9.1.